Vishal Acharya

1. Which of the following functions can be used to fill all null values in a data frame?

fillna()

2. Which attribute of dropna() can be used to select the columns from which null values are to be considered for removing rows?

subset

3. Which of the following pandas functions is used to generate cross tabulation?

crosstab

4. Which of the following DataFrame attributes is used to return one or more specified row(s)?

loc

5. Which of the following attributes can be used to show the number of rows and columns in a Pandas dataframe?

shape

6. Which of the following is not displayed by the Pandas DataFrame info function?

column average

7. Which of the following is not displayed by the Pandas DataFrame describe function?

correlation

Vishal Acharya

8. What does it indicate if the corr() function shows correlation as 1 between any two columns of the DataFrame?

perfect correlation

9. Which of the following is an example of qualitative data?

gender

10. Which of the following is an example of quantitative data?

weight

11. What does DataFrame.dropna(how='all') do?

drops those rows from the DataFrame which contain all null values

12. A data point that differs significantly from other observations is known as

outlier

13. Which of following pandas functions can be used to display the specified number of rows from the beginning of the dataset?

head()

14. Which of following pandas functions can be used to display the specified number of rows from the end of the dataset?

tail()

Vishal Acharya

15. Which of the following represents each data sample as polyline connecting parallel lines where each parallel line represents an attribute of that data sample?

parallel coordinates

16. "What is the output of the code shown below?

In [2]:
```
import pandas as pd
import numpy as np
df=pd.DataFrame([[0,1.0,2.0,np.nan,5],[2.0,0,1.0,5.0,np.nan],[5.0,0,1.0,np.nan,5.0]])
df.dropna()
print(df.loc[1,3])
```

```
5.0
```

17 "What is the output of the code shown below?

In [3]:
```
import pandas as pd
import numpy as np
df=pd.DataFrame([[0,1,2,np.nan,5],[2,0,1,5,np.nan],[5,0,1,np.nan,5],[2,0,1,np.nan,np.nan]])
df=df.drop_duplicates(subset=[1,2])
df=df.drop_duplicates(subset=[4])
df.dropna(thresh=2,axis=1)
print(df.shape)
```

```
(2, 5)
```

18. What type of Error the following code produces?

In [4]:
```
import pandas as pnd
pnd.Series([1,2], index= ['a','b','c'])
```

Vishal Acharya

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[4], line 2
      1 import pandas as pnd
----> 2 pnd.Series([1,2], index= ['a','b','c'])

File ~\anaconda3\Lib\site-packages\pandas\core\series.py:578, in Series.__init__(self, data, index, dtype, name, copy, fastpat
h)
    576     index = default_index(len(data))
    577 elif is_list_like(data):
--> 578     com.require_length_match(data, index)
    580 # create/copy the manager
    581 if isinstance(data, (SingleBlockManager, SingleArrayManager)):

File ~\anaconda3\Lib\site-packages\pandas\core\common.py:573, in require_length_match(data, index)
    569 """
    570 Check the length of data matches the length of the index.
    571 """
    572 if len(data) != len(index):
--> 573     raise ValueError(
    574         "Length of values "
    575         f"({len(data)}) "
    576         "does not match length of index "
    577         f"({len(index)})"
    578     )

ValueError: Length of values (2) does not match length of index (3)
```

19. To remove multiple values from the Pandas dataframe and to keep only the first occurrence values, what will be the correct syntax?

df.drop_duplicates()

20. From a Pandas series ‘s’, if we need to extract indices (1,5,7,12) what will be the syntax used?

s [[1,5,7,12]]

Vishal Acharya

In [10]:
```
import pandas as pd

s = pd.Series([10,20,30,40,50,60,70,80,90,100,110,120,130])

result = s [[1,5,7,12]]

print(result)
```

```
1      20
5      60
7      80
12    130
dtype: int64
```

21 What is the method used to calculate the mean of a numeric column in a DataFrame ?

mean()

22. "What is the output of the below code?

In [13]:
```
import pandas as pd
import numpy as np
df=pd.DataFrame({"a":[1,2,np.nan,3,4],"b":[1,5,np.nan,2,1]})
df=df.drop_duplicates(subset="b")
df.dropna()
df.fillna(20,inplace=True)
print(df.shape[0])
```

```
4
```

23. What is the output of the below code?

In [15]:
```
import pandas as pd
import numpy as np
df=pd.DataFrame([[1,2,3,4,5],[2,1,3,4,5],[np.nan,np.nan,np.nan,np.nan,np.nan]])
df.dropna(thresh=3,axis=1,inplace=True)
print(df.shape[1])
```

Vishal Acharya

```
0
```

24 What is the output of the below code?

In [17]:
```
import pandas as pd
import numpy as np
df=pd.DataFrame([[1,2,3,4,5],[2,1,3,4,5],[np.nan,np.nan,np.nan,np.nan,np.nan]])
df.drop(1,inplace=True)
df=df.dropna()
print(df.shape[0])
```

```
1
```

25 What will be the output of following code?

In [18]:
```
import pandas as pd
data = {
    "Student": ["A","B","C","D","E"],
    "Math": [78, 90, 67, 88, 90],
    "Science": [85, 92, 70, 88, 85]
}
df = pd.DataFrame(data)
result = df.sort_values(by=["Math","Science"], ascending=[False,True])
print(result.head(1)["Student"].values[0])
```

```
E
```

What will be the output of following code?

In [19]:
```
import pandas as pd
df = pd.DataFrame({
    "B":[1,2],
    "A":[3,4],
    "C":[5,6]
})
df2 = df.sort_index(axis=1)
print(df2.columns[0])
```

```
A
```

Vishal Acharya

27 What will be the output of following code?

In [20]:
```
import pandas as pd
df = pd.DataFrame({
    "Price":[100,200,150],
    "Qty":[2,3,4]
})
df["Total"] = df.apply(lambda x: x["Price"]*x["Qty"], axis=1)
print(df["Total"].sum())
```

```
1400
```

28 What will be the output of following code?

In [21]:
```
import pandas as pd
df = pd.DataFrame({
    "Marks":[45.7, 78.2, 90.9]
})
df["Marks"] = df["Marks"].astype(int)
print(df["Marks"].sum())
```

```
213
```

29 What will be the output of following code?

In [22]:
```
import pandas as pd
df = pd.DataFrame({
    "City":["A","B","C","A","B"]
})
df2 = df.replace({"A":"X","B":"Y"})
print(df2["City"].unique())
```

```
['X' 'Y' 'C']
```

30 What will be the output of following code?

Vishal Acharya

In [23]:
```
import pandas as pd
df1 = pd.DataFrame({"A":[1,2]})
df2 = pd.DataFrame({"B":[3,4]})
result = pd.concat([df1, df2], axis=0)
print(result.shape)
```

```
(4, 2)
```

31. What will be the output of following code?

In [24]:
```
import pandas as pd
df1 = pd.DataFrame({"A":[1,2]})
df2 = pd.DataFrame({"A":[3,4]})
result = pd.concat([df1,df2], ignore_index=True)
print(result.index[3])
```

```
3
```

32 What will be the output of following code?

In [26]:
```
import pandas as pd
df1 = pd.DataFrame({
    "ID":[1,2,3],
    "Name":["A","B","C"]
})
df2 = pd.DataFrame({
    "ID":[2,3,4],
    "Marks":[80,90,70]
})
result = pd.merge(df1, df2, how="inner", on="ID")
print(len(result))
```

```
2
```

33 What will be the output of following code?

In [27]:
```
import pandas as pd
df1 = pd.DataFrame({
```

Vishal Acharya

```
    "ID":[1,2,3],
    "Name":["A","B","C"]
})
df2 = pd.DataFrame({
    "ID":[2],
    "Marks":[80]
})
result = pd.merge(df1, df2, how="left", on="ID")
print(result["Marks"].isnull().sum())
```

```
2
```

34 What will be the output of following code?

In [28]:
```
import pandas as pd
df = pd.DataFrame({
    "Team":["X","Y","X","Y","X"],
    "Runs":[50,60,70,80,90]
})
result = df.groupby("Team")["Runs"].mean()
print(result["X"])
```

```
70.0
```

35 What will be the output of following code?

In [29]:
```
import pandas as pd
df = pd.DataFrame({
    "Dept":["IT","HR","IT","HR","IT","Finance"]
})
print(df.groupby("Dept").size()["IT"])
```

```
3
```

36 What will be the output of following code?

In [30]:
```
import pandas as pd
df = pd.DataFrame({
    "Category":["A","A","B","B"],
```

```
    "Value":[10,20,30,40]
})
result = df.groupby("Category").agg({"Value":["sum","max"]})
print(result.loc["B",("Value","sum")])
```

```
70
```

37 What will be the output of following code?

In [31]:
```
import pandas as pd
df = pd.DataFrame({
    "City":["A","A","B","C","C","C"]
})
print(df.groupby("City").nunique().shape[0])
```

```
3
```

38 What will be the output of following code?

In [32]:
```
import pandas as pd
df = pd.DataFrame({
    "Team":["X","X","Y","Y"],
    "Score":[10,20,30,40]
})
result = df.groupby("Team").nth(1)
print(result.loc["X"])
```

Vishal Acharya

Vishal Acharya

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.get_loc(self, key)
   3811 try:
-> 3812     return self._engine.get_loc(casted_key)
   3813 except KeyError as err:

File pandas/_libs/index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:175, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index_class_helper.pxi:70, in pandas._libs.index.Int64Engine._check_type()

KeyError: 'X'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[32], line 7
      2 df = pd.DataFrame({
      3     "Team":["X","X","Y","Y"],
      4     "Score":[10,20,30,40]
      5 })
      6 result = df.groupby("Team").nth(1)
----> 7 print(result.loc["X"])

File ~\anaconda3\Lib\site-packages\pandas\core\indexing.py:1192, in _LocationIndexer.__getitem__(self, key)
   1190 maybe_callable = com.apply_if_callable(key, self.obj)
   1191 maybe_callable = self._check_deprecated_callable_usage(key, maybe_callable)
-> 1192 return self._getitem_axis(maybe_callable, axis=axis)

File ~\anaconda3\Lib\site-packages\pandas\core\indexing.py:1432, in _LocIndexer._getitem_axis(self, key, axis)
   1430 # fall thru to straight lookup
   1431 self._validate_key(key, axis)
-> 1432 return self._get_label(key, axis=axis)

File ~\anaconda3\Lib\site-packages\pandas\core\indexing.py:1382, in _LocIndexer._get_label(self, label, axis)
   1380 def _get_label(self, label, axis: AxisInt):
   1381     # GH#5567 this will fail if the label is not present in the axis.
-> 1382     return self.obj.xs(label, axis=axis)
```

Vishal Acharya

```
File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:4323, in NDFrame.xs(self, key, axis, level, drop_level)
   4321             new_index = index[loc]
   4322 else:
-> 4323     loc = index.get_loc(key)
   4325     if isinstance(loc, np.ndarray):
   4326         if loc.dtype == np.bool_:

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3819, in Index.get_loc(self, key)
   3814     if isinstance(casted_key, slice) or (
   3815         isinstance(casted_key, abc.Iterable)
   3816         and any(isinstance(x, slice) for x in casted_key)
   3817     ):
   3818         raise InvalidIndexError(key)
-> 3819     raise KeyError(key) from err
   3820 except TypeError:
   3821     # If we have a listlike key, _check_indexing_error will raise
   3822     #  InvalidIndexError. Otherwise we fall through and re-raise
   3823     #  the TypeError.
   3824     self._check_indexing_error(key)

KeyError: 'X'
```

39 What will be the output of following code?

In [33]:
```
import pandas as pd
df = pd.DataFrame({
    "Age":[22,25,30,35,40],
    "Salary":[30000,40000,50000,60000,70000]
})
result = df[(df["Age"]>25) & (df["Salary"]>=50000)]
print(len(result))
```

```
3
```

40 What will be the output of following code?

In [34]:
```
import pandas as pd
import numpy as np
df = pd.DataFrame({
```

Vishal Acharya

```
    "A":[1,2,3],
    "B":[4,5,6]
})
result = df.apply(sum, axis=0)
print(result["A"])
```

```
6
```

41 What will be the output of following code?

In [35]:
```
import pandas as pd
df = pd.DataFrame({
    "Team":["A","A","B","B"],
    "Score":[10,20,30,40]
})
result = df.groupby("Team").sample(1)
print(len(result))
```

```
2
```

42 What will be the output of following code?

In [36]:
```
import pandas as pd
df = pd.DataFrame({
    "Dept":["IT","HR","IT","Finance"]
})
groups = df.groupby("Dept").groups
print(groups["IT"])
```

```
Index([0, 2], dtype='int64')
```

43 What will be the output of following code?

In [37]:
```
import pandas as pd
df = pd.DataFrame({
    "Name":["A","B","C","D","E"],
    "Score":[90,90,85,90,85],
    "Age":[20,22,21,19,23]
})
```

```
result = df.sort_values(by=["Score","Age"], ascending=[False,True]).reset_index(drop=True)
print(result.loc[1, "Name"])
```

```
A
```

44 What will be the output of following code?

In [38]:
```
import pandas as pd
df = pd.DataFrame({
    "C":[1,2],
    "A":[3,4],
    "B":[5,6]
})
df = df.drop("B", axis=1)
df2 = df.sort_index(axis=1)
print(list(df2.columns))
```

```
['A', 'C']
```

45 What will be the output of following code?

In [39]:
```
import pandas as pd
df = pd.DataFrame({"A":[1.9,2.1,3.7]})
df["A"] = df["A"].astype(int)
print(df["A"].mean())
```

```
2.0
```

46 What will be the output of following code?

In [40]:
```
import pandas as pd
df = pd.DataFrame({"A":[1,2,3,1,2]})
df2 = df.replace({1:100,2:200})
print(df2["A"].sum())
```

```
603
```

47 What will be the output of following code?

Vishal Acharya

Vishal Acharya

In [41]:
```
df1 = pd.DataFrame({"A":[1,2]}, index=[0,1])
df2 = pd.DataFrame({"A":[3,4]}, index=[1,2])

result = pd.concat([df1,df2])
print(result.loc[2]["A"])
```

4

48 What will be the output of following code?

In [42]:
```
import pandas as pd
df1 = pd.DataFrame({"A":[1,2,3]})
df2 = pd.DataFrame({"B":[4,5]})
result = pd.concat([df1,df2], axis=1)
print(result.isnull().sum().sum())
```

1

49 What will be the output of following code?

In [43]:
```
import pandas as pd
df1 = pd.DataFrame({"ID":[1,2], "Score":[80,90]})
df2 = pd.DataFrame({"ID":[1,2], "Score":[85,95]})
result = pd.merge(df1,df2,on="ID",suffixes=("_X","_Y"))
print(result.loc[0,"Score_Y"])
```

85

50 What will be the output of following code?

In [44]:
```
import pandas as pd
df1 = pd.DataFrame({"ID":[1,2,3]})
df2 = pd.DataFrame({"ID":[2,3,4]})
result = pd.merge(df1,df2,on="ID",how="outer",indicator=True)
print((result["_merge"]=="both").sum())
```

2

Vishal Acharya

51 What will be the output of following code?

In [45]:
```
import pandas as pd
df = pd.DataFrame({
    "Dept":["IT","IT","HR"],
    "Salary":[100,200,300]
})
result = df.groupby("Dept",as_index=False).sum()
print(type(result))
```

```
<class 'pandas.core.frame.DataFrame'>
```

52 What will be the output of following code?

In [46]:
```
import pandas as pd

df = pd.DataFrame({
    "Cat":["A","A","B","B"],
    "Val":[5,15,10,20]
})

result = df.groupby("Cat").agg({"Val":["min","max","mean"]})
print(result.loc["A",("Val","mean")])
```

```
10.0
```

53 What will be the output of following code?

In [47]:
```
import pandas as pd
df = pd.DataFrame({
    "Team":["X","X","Y"],
    "Score":[10,20,30]
})
result = df.groupby("Team").apply(lambda x: x["Score"].max()-x["Score"].min())
print(result["X"])
```

```
10
```

Vishal Acharya

```
C:\Users\VISHAL\AppData\Local\Temp\ipykernel_23480\13691750.py:6: FutureWarning: DataFrameGroupBy.apply operated on the groupin
g columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operat
ion. Either pass `include_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to sil
ence this warning.
  result = df.groupby("Team").apply(lambda x: x["Score"].max()-x["Score"].min())
```

54 What will be the output of following code?

In [48]:
```
import pandas as pd
df = pd.DataFrame({
    "Dept":["IT","IT","HR"],
    "City":["A","A","B"]
})
result = df.groupby("Dept").nunique()
print(result.loc["IT","City"])
```

```
1
```

55 Create a Pandas DataFrame from the following table and write code to remove all rows from this table containing at least one NaN value

In [49]:
```
import pandas as pd
import numpy as np

# Create DataFrame
data = {
    "name": ["William", "Emma", "Sofia", "Markus", "Edward", "Thomas", "Ethan", np.nan, "Arun", "Anika", "Paulo"],
    "region": [np.nan, "North", "East", np.nan, "West", "West", "South", np.nan, "West", "East", "South"],
    "sales": [50000.0, 52000.0, np.nan, np.nan, 42000.0, 72000.0, 49000.0, np.nan, 67000.0, 65000.0, 67000.0],
    "expenses": [42000.0, 43000.0, np.nan, np.nan, 38000.0, 39000.0, 42000.0, np.nan, 39000.0, 50000.0, 45000.0]
}

df = pd.DataFrame(data)

# Remove rows containing at least one NaN value
df_clean = df.dropna()

print(df_clean)
```

Vishal Acharya

```
      name region    sales  expenses
1     Emma  North  52000.0   43000.0
4   Edward   West  42000.0   38000.0
5   Thomas   West  72000.0   39000.0
6    Ethan  South  49000.0   42000.0
8     Arun   West  67000.0   39000.0
9    Anika   East  65000.0   50000.0
10   Paulo  South  67000.0   45000.0
```

56 Create a Pandas DataFrame from the following table and write code to remove all rows from this table only if all of their values are NaN

In [51]:

```
import pandas as pd
import numpy as np

# Create DataFrame
data = {
    "name": ["William", "Emma", "Sofia", "Markus", "Edward", "Thomas", "Ethan", np.nan, "Arun", "Anika", "Paulo"],
    "region": [np.nan, "North", "East", np.nan, "West", "West", "South", np.nan, "West", "East", "South"],
    "sales": [50000.0, 52000.0, np.nan, np.nan, 42000.0, 72000.0, 49000.0, np.nan, 67000.0, 65000.0, 67000.0],
    "expenses": [42000.0, 43000.0, np.nan, np.nan, 38000.0, 39000.0, 42000.0, np.nan, 39000.0, 50000.0, 45000.0]
}

df = pd.DataFrame(data)

# Remove rows containing at least one NaN value
df_clean = df.dropna(how="all")

print(df_clean)
```

Vishal Acharya

```
       name region    sales  expenses
0   William    NaN  50000.0   42000.0
1      Emma  North  52000.0   43000.0
2     Sofia   East      NaN       NaN
3    Markus    NaN      NaN       NaN
4    Edward   West  42000.0   38000.0
5    Thomas   West  72000.0   39000.0
6     Ethan  South  49000.0   42000.0
8      Arun   West  67000.0   39000.0
9     Anika   East  65000.0   50000.0
10    Paulo  South  67000.0   45000.0
```

57 Create a Pandas DataFrame from the following table and write code to drop all columns containing NaN

In [52]:

```python
import pandas as pd
import numpy as np

# Create DataFrame
data = {
    "name": ["William", "Emma", "Sofia", "Markus", "Edward", "Thomas", "Ethan", np.nan, "Arun", "Anika", "Paulo"],
    "region": [np.nan, "North", "East", np.nan, "West", "West", "South", np.nan, "West", "East", "South"],
    "sales": [50000.0, 52000.0, np.nan, np.nan, 42000.0, 72000.0, 49000.0, np.nan, 67000.0, 65000.0, 67000.0],
    "expenses": [42000.0, 43000.0, np.nan, np.nan, 38000.0, 39000.0, 42000.0, np.nan, 39000.0, 50000.0, 45000.0]
}

df = pd.DataFrame(data)

# Drop columns containing any NaN values
df_clean = df.dropna(axis=1)

print(df_clean)
```

```
Empty DataFrame
Columns: []
Index: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

58 Write Python code to remove outliers from any given DataFrame.

Vishal Acharya

In [53]:
```python
import pandas as pd
import numpy as np

def remove_outliers(df):

    # Select only numeric columns
    num_cols = df.select_dtypes(include=np.number).columns

    for col in num_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]

    return df

# Example usage
data = {
    "Age": [21, 22, 23, 24, 120],
    "Salary": [30000, 32000, 35000, 34000, 900000]
}

df = pd.DataFrame(data)

clean_df = remove_outliers(df)

print(clean_df)
```

```
   Age  Salary
0   21   30000
1   22   32000
2   23   35000
3   24   34000
```

59 Consider the following data:

Vishal Acharya

data = {

"A": ["TeamA", "TeamB", "TeamB", "TeamC", "TeamA"],

"B": [50, 40, 40, 30, 50],

"C": [True, False, False, False, True]

}

Convert this to a Pandas DataFrame and remove duplicate rows from it. Reset index values.

In [14]:
```
import pandas as pd

# Create a DataFrame from the data
data = {
  "A": ["TeamA", "TeamB", "TeamB", "TeamC", "TeamA"],
  "B": [50, 40, 40, 30, 50],
  "C": [True, False, False, False, True]
}

df = pd.DataFrame(data)

# Remove duplicate rows and reset index
df = df.drop_duplicates().reset_index(drop=True)

# Print the DataFrame
print(df)

df.set_index('A',inplace=True)
df
```

```
       A   B      C
0  TeamA  50   True
1  TeamB  40  False
2  TeamC  30  False
```

Out[14]:

A	B	C
TeamA	50	True
TeamB	40	False
TeamC	30	False

60 Consider the following autompg dataset: Write Python code to convert it to a DataFrame and remove mpg and cylinders columns from it

In [54]:

```
import pandas as pd

# Load the dataset into a DataFrame
df = pd.read_csv("auto-mpg.csv")

# Display the first few rows of the DataFrame to understand its structure
print("Original DataFrame:")
print(df.head())

# Remove the 'mpg' and 'cylinders' columns
df = df.drop(columns=['mpg', 'cylinders'])

# Display the first few rows of the updated DataFrame
print("\nDataFrame after removing 'mpg' and 'cylinders' columns:")
print(df.head())
```

Vishal Acharya

Vishal Acharya

```
Original DataFrame:
    mpg  cylinders  displacement horsepower  weight  acceleration  model year  \
0  18.0          8         307.0        130    3504          12.0          70
1  15.0          8         350.0        165    3693          11.5          70
2  18.0          8         318.0        150    3436          11.0          70
3  16.0          8         304.0        150    3433          12.0          70
4  17.0          8         302.0        140    3449          10.5          70

   origin                   car name
0       1  chevrolet chevelle malibu
1       1          buick skylark 320
2       1         plymouth satellite
3       1              amc rebel sst
4       1                ford torino

DataFrame after removing 'mpg' and 'cylinders' columns:
   displacement horsepower  weight  acceleration  model year  origin  \
0         307.0        130    3504          12.0          70       1
1         350.0        165    3693          11.5          70       1
2         318.0        150    3436          11.0          70       1
3         304.0        150    3433          12.0          70       1
4         302.0        140    3449          10.5          70       1

                   car name
0  chevrolet chevelle malibu
1          buick skylark 320
2         plymouth satellite
3              amc rebel sst
4                ford torino
```

61. Use the file heights_weights.csv (https://raw.githubusercontent.com/Jovita7/Data-Analysis-and-Visualization/main/heights_weights.csv) which contains 10000 non-null values for heights and weights. The Male column shows 1 if the person is a Male and 0 if the person is a Female.

1. Convert this file into a pandas Data Frame. (0.5 marks)
2. Display basic information like memory and data types for this data frame. (0.5 marks)
3. Display basic statistics like mean, std, quartiles, etc. for this data frame. (0.5 marks)
4. Create a correlation table for the data frame and comment about what kind of correlation is there between Height and Weight. (0.5 marks)

5. Do Height and Weight contain any outliers? (1 mark)

1. Convert this file into a pandas Data Frame.

In [61]:
```
import pandas as pd



# Load the dataset into a DataFrame
df = pd.read_csv("heights_weights.csv")

# Display the first few rows of the DataFrame to verify the loading
print("Original DataFrame:")
print(df.head())
```

```
Original DataFrame:
      Height      Weight  Male
0  73.847017  241.893563     1
1  68.781904  162.310473     1
2  74.110105  212.740856     1
3  71.730978  220.042470     1
4  69.881796  206.349801     1
```

2. Display basic information like memory and data types for this data frame

In [62]:
```
# Display basic information about the DataFrame
print("\nBasic Information:")
print(df.info())
```

```
Basic Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Height  10000 non-null  float64
 1   Weight  10000 non-null  float64
 2   Male    10000 non-null  int64
dtypes: float64(2), int64(1)
memory usage: 234.5 KB
None
```

3. Display basic statistics like mean, std, quartiles, etc. for this data frame

Vishal Acharya

In [63]:
```python
# Display basic statistics for the DataFrame
print("\nBasic Statistics:")
print(df.describe())
```

```
Basic Statistics:
             Height        Weight          Male
count  10000.000000  10000.000000  10000.000000
mean      66.367560    161.440357      0.500000
std        3.847528     32.108439      0.500025
min       54.263133     64.700127      0.000000
25%       63.505620    135.818051      0.000000
50%       66.318070    161.212928      0.500000
75%       69.174262    187.169525      1.000000
max       78.998742    269.989699      1.000000
```

4. Create a correlation table for the data frame and comment about what kind of correlation is there between Height and Weight

In [65]:
```python
# Create a correlation table
correlation_matrix = df.corr(numeric_only=True)

# Display the correlation table
print("\nCorrelation Table:")
print(correlation_matrix)

# Comment about the correlation between Height and Weight
height_weight_corr = correlation_matrix.loc['Height', 'Weight']
print(f"\nCorrelation between Height and Weight: {height_weight_corr}")
if height_weight_corr > 0:
    print("There is a positive correlation between Height and Weight.")
elif height_weight_corr < 0:
    print("There is a negative correlation between Height and Weight.")
else:
    print("There is no correlation between Height and Weight.")
```

Vishal Acharya

```
Correlation Table:
          Height    Weight      Male
Height  1.000000  0.924756  0.691072
Weight  0.924756  1.000000  0.796723
Male    0.691072  0.796723  1.000000

Correlation between Height and Weight: 0.9247562987409196
There is a positive correlation between Height and Weight.
```

5. Do Height and Weight contain any outliers?

In [66]:

```python
# Function to detect outliers using the IQR method
def detect_outliers(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df[(df[column] < lower_bound) | (df[column] > upper_bound)]
    return outliers

# Detect outliers in Height
height_outliers = detect_outliers(df, 'Height')
print(f"\nNumber of outliers in Height: {len(height_outliers)}")

# Detect outliers in Weight
weight_outliers = detect_outliers(df, 'Weight')
print(f"Number of outliers in Weight: {len(weight_outliers)}")

# Display some of the outliers if they exist
if not height_outliers.empty:
    print("\nOutliers in Height:")
    print(height_outliers.head())

if not weight_outliers.empty:
    print("\nOutliers in Weight:")
    print(weight_outliers.head())
```

Vishal Acharya

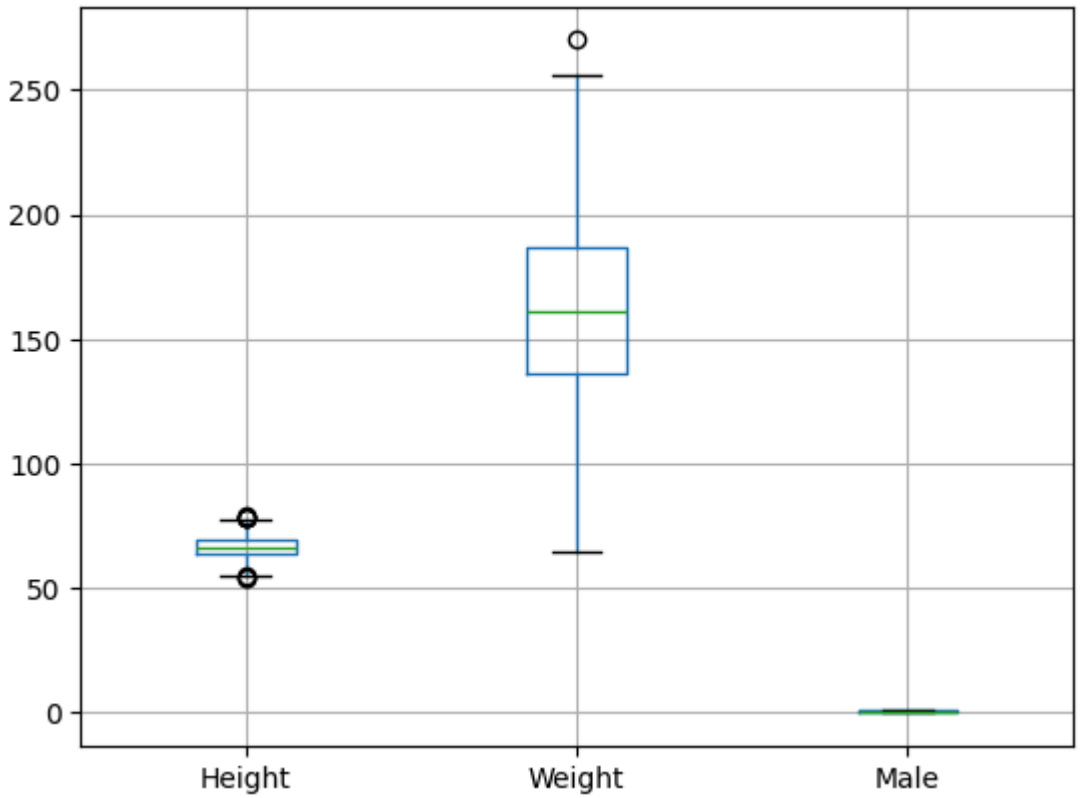
```
Number of outliers in Height: 8
Number of outliers in Weight: 1

Outliers in Height:
         Height      Weight  Male
994   78.095867  255.690835     1
1317  78.462053  227.342565     1
2014  78.998742  269.989699     1
3285  78.528210  253.889004     1
3757  78.621374  245.733783     1

Outliers in Weight:
         Height      Weight  Male
2014  78.998742  269.989699     1
```

In [67]:
```
df.boxplot()
```

Out[67]:
```
<Axes: >
```

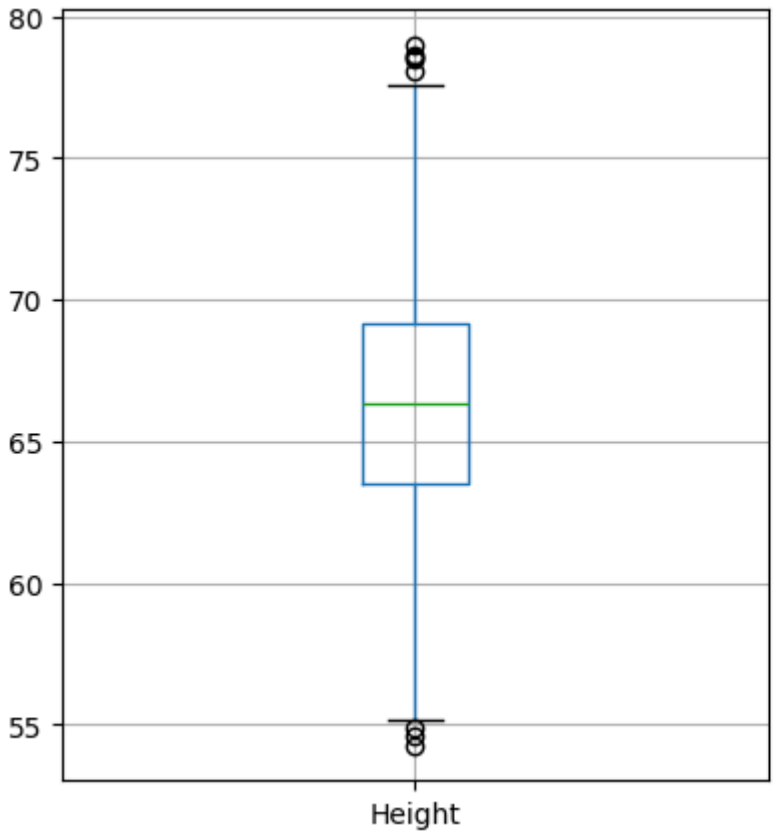
Vishal Acharya

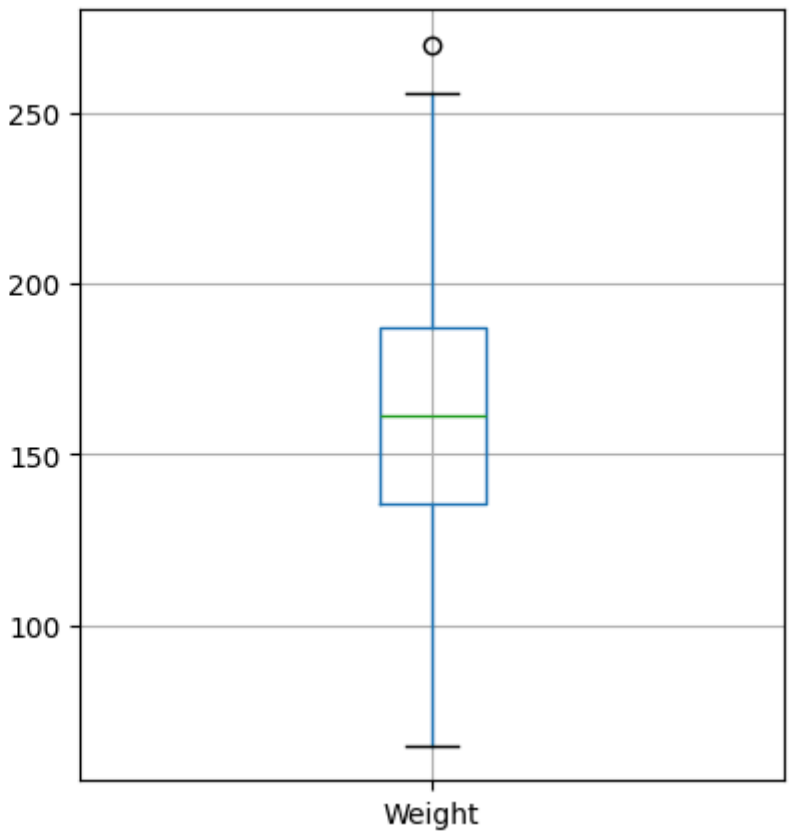
Vishal Acharya



In [68]:
```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(1, 2, figsize=(10,5))
df.boxplot(column=['Height'], ax=ax[0])
df.boxplot(column=['Weight'], ax=ax[1])
plt.show()
```

Vishal Acharya





62 Use the file ipl-matches.csv which contains data of all the IPL matches from year 2008 to 2022. Read this csv file and display the basic information like memory and data types for this data frame. Write python code for the following cases:

1. List out all matches gone in superover.
2. How Many Matches won by Chennai Super Kings at Kolkata.
3. In How Many Matches MS Dhoni is Player of Match Vs Mumbai Indians.
4. Display list of all matches in which Gujarat Titans won the Toss and Elected to Bat and won the match.
5. Display list of all matches won by Gujarat Titans.

List out all matches gone in superover.

Vishal Acharya

In [71]:
```python
import pandas as pd

# Load the dataset into a DataFrame
df = pd.read_csv("ipl-matches.csv")

# Display basic information about the DataFrame
print("\nBasic Information:")
print(df.info())
```

```
Basic Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 950 entries, 0 to 949
Data columns (total 20 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   ID               950 non-null    int64
 1   City             899 non-null    object
 2   Date             950 non-null    object
 3   Season           950 non-null    object
 4   MatchNumber      950 non-null    object
 5   Team1            950 non-null    object
 6   Team2            950 non-null    object
 7   Venue            950 non-null    object
 8   TossWinner       950 non-null    object
 9   TossDecision     950 non-null    object
 10  SuperOver        946 non-null    object
 11  WinningTeam      946 non-null    object
 12  WonBy            950 non-null    object
 13  Margin           932 non-null    float64
 14  method           19 non-null     object
 15  Player_of_Match  946 non-null    object
 16  Team1Players     950 non-null    object
 17  Team2Players     950 non-null    object
 18  Umpire1          950 non-null    object
 19  Umpire2          950 non-null    object
dtypes: float64(1), int64(1), object(18)
memory usage: 148.6+ KB
None
```

In [72]:
```
# Display the first few rows of the DataFrame to understand its structure
print("\nFirst few rows of the DataFrame:")
print(df.head())
```

Vishal Acharya

Vishal Acharya

```
First few rows of the DataFrame:
        ID       City        Date Season  MatchNumber  \
0  1312200  Ahmedabad  2022-05-29   2022        Final
1  1312199  Ahmedabad  2022-05-27   2022  Qualifier 2
2  1312198    Kolkata  2022-05-25   2022   Eliminator
3  1312197    Kolkata  2022-05-24   2022  Qualifier 1
4  1304116     Mumbai  2022-05-22   2022           70

                        Team1                 Team2  \
0             Rajasthan Royals        Gujarat Titans
1  Royal Challengers Bangalore      Rajasthan Royals
2  Royal Challengers Bangalore  Lucknow Super Giants
3             Rajasthan Royals        Gujarat Titans
4          Sunrisers Hyderabad          Punjab Kings

                             Venue            TossWinner TossDecision  \
0  Narendra Modi Stadium, Ahmedabad      Rajasthan Royals          bat
1  Narendra Modi Stadium, Ahmedabad      Rajasthan Royals        field
2             Eden Gardens, Kolkata  Lucknow Super Giants        field
3             Eden Gardens, Kolkata        Gujarat Titans        field
4          Wankhede Stadium, Mumbai   Sunrisers Hyderabad          bat

  SuperOver                  WinningTeam    WonBy  Margin method  \
0         N               Gujarat Titans  Wickets     7.0    NaN
1         N             Rajasthan Royals  Wickets     7.0    NaN
2         N  Royal Challengers Bangalore     Runs    14.0    NaN
3         N               Gujarat Titans  Wickets     7.0    NaN
4         N                 Punjab Kings  Wickets     5.0    NaN

  Player_of_Match                                       Team1Players  \
0      HH Pandya  ['YBK Jaiswal', 'JC Buttler', 'SV Samson', 'D ...
1     JC Buttler  ['V Kohli', 'F du Plessis', 'RM Patidar', 'GJ ...
2     RM Patidar  ['V Kohli', 'F du Plessis', 'RM Patidar', 'GJ ...
3      DA Miller  ['YBK Jaiswal', 'JC Buttler', 'SV Samson', 'D ...
4   Harpreet Brar  ['PK Garg', 'Abhishek Sharma', 'RA Tripathi', ...

                                       Team2Players        Umpire1  \
0  ['WP Saha', 'Shubman Gill', 'MS Wade', 'HH Pan...    CB Gaffaney
1  ['YBK Jaiswal', 'JC Buttler', 'SV Samson', 'D ...    CB Gaffaney
2  ['Q de Kock', 'KL Rahul', 'M Vohra', 'DJ Hooda...  J Madanagopal
3  ['WP Saha', 'Shubman Gill', 'MS Wade', 'HH Pan...   BNJ Oxenford
```

```
4  ['JM Bairstow', 'S Dhawan', 'M Shahrukh Khan',...   AK Chaudhary

        Umpire2
0    Nitin Menon
1    Nitin Menon
2       MA Gough
3      VK Sharma
4  NA Patwardhan
```

In [73]:

```python
# Filter matches where SuperOver is 1
super_over_matches = df[df['SuperOver'] == "Y"]
print("super_over_matches",super_over_matches.shape[0])

# Display the matches gone in super over
print("\nMatches gone in super over:")
print(super_over_matches[['ID', 'Team1', 'Team2', 'City', 'Date']])
```

Vishal Acharya

Vishal Acharya

```
super_over_matches 14

Matches gone in super over:
          ID                        Team1                        Team2  \
114  1254077               Delhi Capitals          Sunrisers Hyderabad
158  1216512        Kolkata Knight Riders          Sunrisers Hyderabad
159  1216517               Mumbai Indians              Kings XI Punjab
184  1216547  Royal Challengers Bangalore               Mumbai Indians
192  1216493               Delhi Capitals              Kings XI Punjab
203  1178426               Mumbai Indians          Sunrisers Hyderabad
244  1175365        Kolkata Knight Riders               Delhi Capitals
339  1082625                Gujarat Lions               Mumbai Indians
474   829741             Rajasthan Royals              Kings XI Punjab
533   729315        Kolkata Knight Riders             Rajasthan Royals
608   598017  Royal Challengers Bangalore             Delhi Daredevils
621   598004          Sunrisers Hyderabad  Royal Challengers Bangalore
819   419121          Chennai Super Kings              Kings XI Punjab
883   392190        Kolkata Knight Riders             Rajasthan Royals

          City        Date
114    Chennai  2021-04-25
158  Abu Dhabi  2020-10-18
159        NaN  2020-10-18
184        NaN  2020-09-28
192        NaN  2020-09-20
203     Mumbai  2019-05-02
244      Delhi  2019-03-30
339     Rajkot  2017-04-29
474  Ahmedabad  2015-04-21
533  Abu Dhabi  2014-04-29
608  Bangalore  2013-04-16
621  Hyderabad  2013-04-07
819    Chennai  2010-03-21
883  Cape Town  2009-04-23
```

How Many Matches won by Chennai Super Kings at Kolkata.

In [74]:
```
# Filter matches won by Chennai Super Kings at Kolkata
csk_kolkata_wins = df[(df['WinningTeam'] == 'Chennai Super Kings') & (df['City'] == 'Kolkata')]

# Count the number of matches
```

```
csk_kolkata_wins_count = csk_kolkata_wins.shape[0]
print(f"\nNumber of matches won by Chennai Super Kings at Kolkata: {csk_kolkata_wins_count}")
```

```
Number of matches won by Chennai Super Kings at Kolkata: 5
```

In How Many Matches MS Dhoni is Player of Match Vs Mumbai Indians.

In [75]:
```
# Filter matches where MS Dhoni was the Player of the Match and Mumbai Indians were one of the teams
dhoni_pom_vs_mi = df[(df['Player_of_Match'] == 'MS Dhoni') &
                     ((df['Team1'] == 'Mumbai Indians') | (df['Team2'] == 'Mumbai Indians'))]

# Count the number of such matches
dhoni_pom_vs_mi_count = dhoni_pom_vs_mi.shape[0]
print(f"\nNumber of matches where MS Dhoni is Player of Match Vs Mumbai Indians: {dhoni_pom_vs_mi_count}")
dhoni_pom_vs_mi
```

```
Number of matches where MS Dhoni is Player of Match Vs Mumbai Indians: 1
```

Out[75]:

	ID	City	Date	Season	MatchNumber	Team1	Team2	Venue	TossWinner	TossDecision	SuperOver	WinningTea
630	548379	Bangalore	2012-05-23	2012	Elimination Final	Chennai Super Kings	Mumbai Indians	M Chinnaswamy Stadium	Mumbai Indians	field	N	Chennai Sup Kin

Display list of all matches in which Gujarat Titans won the Toss and Elected to Bat and won the match.

In [76]:
```
# Filter matches where Gujarat Titans won the toss, elected to bat, and won the match
gt_toss_bat_wins = df[(df['TossWinner'] == 'Gujarat Titans') &
                      (df['TossDecision'] == 'bat') &
                      (df['WinningTeam'] == 'Gujarat Titans')]

# Display the filtered matches
print("\nMatches where Gujarat Titans won the Toss, Elected to Bat, and won the match:")
print(gt_toss_bat_wins[['ID', 'Team1', 'Team2', 'City', 'Date']])
```

```
Matches where Gujarat Titans won the Toss, Elected to Bat, and won the match:
         ID           Team1                 Team2         City        Date
17  1304103  Gujarat Titans   Lucknow Super Giants         Pune  2022-05-10
39  1304081  Gujarat Titans  Kolkata Knight Riders  Navi Mumbai  2022-04-23
```

Display list of all matches won by Gujarat Titans

Vishal Acharya

Vishal Acharya

In [77]:
```
# Filter matches where Gujarat Titans won the toss, elected to bat, and won the match
gt_toss_bat_wins = df[(df['TossWinner'] == 'Gujarat Titans') &
                      (df['TossDecision'] == 'bat') &
                      (df['WinningTeam'] == 'Gujarat Titans')]

# Display the filtered matches
print("\nMatches where Gujarat Titans won the Toss, Elected to Bat, and won the match:")
print(gt_toss_bat_wins[['ID', 'Team1', 'Team2', 'City', 'Date']])
```

```
Matches where Gujarat Titans won the Toss, Elected to Bat, and won the match:
         ID           Team1                  Team2         City        Date
17  1304103  Gujarat Titans   Lucknow Super Giants         Pune  2022-05-10
39  1304081  Gujarat Titans  Kolkata Knight Riders  Navi Mumbai  2022-04-23
```

63. Use the file spotify.csv

- 1.Convert this file into a pandas Data Frame.
- 2.Display basic information like memory and data types for this data frame.
- 3.Display basic statistics like mean, std, quartiles, etc. for this data frame.
- 4.Create a correlation table for the data frame and comment about what kind of correlation is there between danceability and energy
- 5.Display first five rows for this data frame.
- 6.Display last five rows for this data frame.
- 7.Display the rows between 15 to 39 for this data frame.
- 8.Display the data only for last five rows and last five columns for this data frame.
- 9.Display the shape for this data frame.
- 10.Display the sum of NULL values for all the columns.
- 11.Remove first 3 columns from this Data Frame.
- 12.Remove first 10 rows from this Data Frame.
- 13.After removing first 3 columns and first 10 rows from this data frame find outliers for the column popularity.
- 14.After removing first 3 columns and first 10 rows from this data frame remove outliers for the column energy then display the data frame.
- 15.Display cross tabulation between time_signature and track_genre for actual Data Frame.

1.Convert this file into a pandas Data Frame.

In [78]:
```
df =pd.read_csv("dataset.csv")

df
```

Vishal Acharya

Out[78]:

	Unnamed: 0	track_id	artists	album_name	track_name	popularity	duration_ms	explicit	danceability
0	0	5SuOikwiRyPMVoIQDJUgSV	Gen Hoshino	Comedy	Comedy	73	230666	False	0.676
1	1	4qPNDBW1i3p13qLCt0Ki3A	Ben Woodward	Ghost (Acoustic)	Ghost - Acoustic	55	149610	False	0.420
2	2	1iJBSr7s7jYXzM8EGcbK5b	Ingrid Michaelson;ZAYN	To Begin Again	To Begin Again	57	210826	False	0.438
3	3	6lfxq3CG4xtTiEg7opyCyx	Kina Grannis	Crazy Rich Asians (Original Motion Picture Sou...	Can't Help Falling In Love	71	201933	False	0.266
4	4	5vjLSffimiIP26QG5WcN2K	Chord Overstreet	Hold On	Hold On	82	198853	False	0.618
...	...	...	...	...	...	...	...	...	...
113995	113995	2C3TZjDRiAzdyViavDJ217	Rainy Lullaby	#mindfulness - Soft Rain for Mindful Meditatio...	Sleep My Little Boy	21	384999	False	0.172
113996	113996	1hIz5L4IB9hN3WRYPOCGPw	Rainy Lullaby	#mindfulness - Soft Rain for Mindful Meditatio...	Water Into Light	22	385000	False	0.174
113997	113997	6x8ZfSoqDjuNa5SVP5QjvX	Cesária Evora	Best Of	Miss Perfumado	22	271466	False	0.629
113998	113998	2e6sXL2bYv4bSz6VTdnfLs	Michael W. Smith	Change Your World	Friends	41	283893	False	0.587
113999	113999	2hETkH7cOfqmz3LqZDHZf5	Cesária Evora	Miss Perfumado	Barbincor	22	241826	False	0.526

114000 rows × 21 columns

Vishal Acharya

2.Display basic information like memory and data types for this data frame.

In [79]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 114000 entries, 0 to 113999
Data columns (total 21 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   Unnamed: 0        114000 non-null  int64
 1   track_id          114000 non-null  object
 2   artists           113999 non-null  object
 3   album_name        113999 non-null  object
 4   track_name        113999 non-null  object
 5   popularity        114000 non-null  int64
 6   duration_ms       114000 non-null  int64
 7   explicit          114000 non-null  bool
 8   danceability      114000 non-null  float64
 9   energy            114000 non-null  float64
 10  key               114000 non-null  int64
 11  loudness          114000 non-null  float64
 12  mode              114000 non-null  int64
 13  speechiness       114000 non-null  float64
 14  acousticness      114000 non-null  float64
 15  instrumentalness  114000 non-null  float64
 16  liveness          114000 non-null  float64
 17  valence           114000 non-null  float64
 18  tempo             114000 non-null  float64
 19  time_signature    114000 non-null  int64
 20  track_genre       114000 non-null  object
dtypes: bool(1), float64(9), int64(6), object(5)
memory usage: 17.5+ MB
```

3.Display basic statistics like mean, std, quartiles, etc. for this data frame.

In [80]:
```
df.describe()
```

Vishal Acharya

Out[80]:

	Unnamed: 0	popularity	duration_ms	danceability	energy	key	loudness	mode	speech
count	114000.000000	114000.000000	1.140000e+05	114000.000000	114000.000000	114000.000000	114000.000000	114000.000000	114000.00
mean	56999.500000	33.238535	2.280292e+05	0.566800	0.641383	5.309140	-8.258960	0.637553	0.08
std	32909.109681	22.305078	1.072977e+05	0.173542	0.251529	3.559987	5.029337	0.480709	0.10
min	0.000000	0.000000	0.000000e+00	0.000000	0.000000	0.000000	-49.531000	0.000000	0.00
25%	28499.750000	17.000000	1.740660e+05	0.456000	0.472000	2.000000	-10.013000	0.000000	0.03
50%	56999.500000	35.000000	2.129060e+05	0.580000	0.685000	5.000000	-7.004000	1.000000	0.04
75%	85499.250000	50.000000	2.615060e+05	0.695000	0.854000	8.000000	-5.003000	1.000000	0.08
max	113999.000000	100.000000	5.237295e+06	0.985000	1.000000	11.000000	4.532000	1.000000	0.96

4.Create a correlation table for the data frame and comment about what kind of correlation is there between danceability and energy

In [81]:
```
print("Weak positive correlation: Danceability increases as energy tends to increase.")
df.corr(numeric_only=True)
```

```
Weak positive correlation: Danceability increases as energy tends to increase.
```

Vishal Acharya

Out[81]:

	Unnamed: 0	popularity	duration_ms	explicit	danceability	energy	key	loudness	mode	speechiness	acc
Unnamed: 0	1.000000	0.032142	-0.032743	-0.054736	0.003444	-0.055994	-0.005520	-0.027307	0.005107	-0.084952	
popularity	0.032142	1.000000	-0.007101	0.044082	0.035448	0.001056	-0.003853	0.050423	-0.013931	-0.044927	
duration_ms	-0.032743	-0.007101	1.000000	-0.065263	-0.073426	0.058523	0.008114	-0.003470	-0.035556	-0.062600	
explicit	-0.054736	0.044082	-0.065263	1.000000	0.122507	0.096955	0.004484	0.108588	-0.037212	0.307952	
danceability	0.003444	0.035448	-0.073426	0.122507	1.000000	0.134325	0.036469	0.259077	-0.069219	0.108626	
energy	-0.055994	0.001056	0.058523	0.096955	0.134325	1.000000	0.048006	0.761690	-0.078362	0.142509	
key	-0.005520	-0.003853	0.008114	0.004484	0.036469	0.048006	1.000000	0.038590	-0.135916	0.020418	
loudness	-0.027307	0.050423	-0.003470	0.108588	0.259077	0.761690	0.038590	1.000000	-0.041764	0.060826	
mode	0.005107	-0.013931	-0.035556	-0.037212	-0.069219	-0.078362	-0.135916	-0.041764	1.000000	-0.046532	
speechiness	-0.084952	-0.044927	-0.062600	0.307952	0.108626	0.142509	0.020418	0.060826	-0.046532	1.000000	
acousticness	0.076840	-0.025472	-0.103788	-0.094403	-0.171533	-0.733906	-0.040937	-0.589803	0.095553	-0.002186	
instrumentalness	-0.070286	-0.095139	0.124371	-0.103404	-0.185606	-0.181879	-0.006823	-0.433477	-0.049955	-0.089616	
liveness	0.033639	-0.005387	0.010321	0.032549	-0.131617	0.184796	-0.001600	0.076899	0.014012	0.205219	
valence	0.053111	-0.040534	-0.154479	-0.003381	0.477341	0.258934	0.034103	0.279848	0.021953	0.036635	
tempo	-0.025824	0.013205	0.024346	-0.002816	-0.050450	0.247851	0.010917	0.212446	0.000566	0.017273	
time_signature	-0.021115	0.031073	0.018225	0.038386	0.207218	0.187126	0.015065	0.191992	-0.024092	-0.000011	

5.Display first five rows for this data frame.

In [82]:
```
df.head()
```

Vishal Acharya

Out[82]:

	Unnamed: 0	track_id	artists	album_name	track_name	popularity	duration_ms	explicit	danceability	energ
0	0	5SuOikwiRyPMVoIQDJUgSV	Gen Hoshino	Comedy	Comedy	73	230666	False	0.676	0.461
1	1	4qPNDBW1i3p13qLCt0Ki3A	Ben Woodward	Ghost (Acoustic)	Ghost - Acoustic	55	149610	False	0.420	0.166
2	2	1iJBSr7s7jYXzM8EGcbK5b	Ingrid Michaelson;ZAYN	To Begin Again	To Begin Again	57	210826	False	0.438	0.359
3	3	6lfxq3CG4xtTiEg7opyCyx	Kina Grannis	Crazy Rich Asians (Original Motion Picture Sou...	Can't Help Falling In Love	71	201933	False	0.266	0.059
4	4	5vjLSffimiIP26QG5WcN2K	Chord Overstreet	Hold On	Hold On	82	198853	False	0.618	0.443

5 rows × 21 columns

6.Display last five rows for this data frame.

In [83]:
```
df.tail()
```

Vishal Acharya

Out[83]:

	Unnamed: 0	track_id	artists	album_name	track_name	popularity	duration_ms	explicit	danceability	energy
113995	113995	2C3TZjDRiAzdyViavDJ217	Rainy Lullaby	#mindfulness - Soft Rain for Mindful Meditatio...	Sleep My Little Boy	21	384999	False	0.172	0.235
113996	113996	1hIz5L4IB9hN3WRYPOCGPw	Rainy Lullaby	#mindfulness - Soft Rain for Mindful Meditatio...	Water Into Light	22	385000	False	0.174	0.117
113997	113997	6x8ZfSoqDjuNa5SVP5QjvX	Cesária Evora	Best Of	Miss Perfumado	22	271466	False	0.629	0.329
113998	113998	2e6sXL2bYv4bSz6VTdnfLs	Michael W. Smith	Change Your World	Friends	41	283893	False	0.587	0.506
113999	113999	2hETkH7cOfqmz3LqZDHZf5	Cesária Evora	Miss Perfumado	Barbincor	22	241826	False	0.526	0.487

5 rows × 21 columns

7.Display the rows between 15 to 39 for this data frame.

In [84]:
```
df.tail()
```

Vishal Acharya

Out[84]:

	Unnamed: 0	track_id	artists	album_name	track_name	popularity	duration_ms	explicit	danceability	energy
113995	113995	2C3TZjDRiAzdyViavDJ217	Rainy Lullaby	#mindfulness - Soft Rain for Mindful Meditatio...	Sleep My Little Boy	21	384999	False	0.172	0.235
113996	113996	1hIz5L4IB9hN3WRYPOCGPw	Rainy Lullaby	#mindfulness - Soft Rain for Mindful Meditatio...	Water Into Light	22	385000	False	0.174	0.117
113997	113997	6x8ZfSoqDjuNa5SVP5QjvX	Cesária Evora	Best Of	Miss Perfumado	22	271466	False	0.629	0.329
113998	113998	2e6sXL2bYv4bSz6VTdnfLs	Michael W. Smith	Change Your World	Friends	41	283893	False	0.587	0.506
113999	113999	2hETkH7cOfqmz3LqZDHZf5	Cesária Evora	Miss Perfumado	Barbincor	22	241826	False	0.526	0.487

5 rows × 21 columns

8.Display the data only for last five rows and last five columns for this data frame.

In [85]:
```
df.head()+df.tail()
```

Vishal Acharya

Out[85]:

	Unnamed: 0	track_id	artists	album_name	track_name	popularity	duration_ms	explicit	danceability	energy	...	loudness	moc
0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	Na
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	Na
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	Na
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	Na
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	Na
113995	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	Na
113996	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	Na
113997	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	Na
113998	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	Na
113999	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	Na

10 rows × 21 columns

9.Display the shape for this data frame.

In [86]:
```
df.shape
```

Out[86]:
```
(114000, 21)
```

10.Display the sum of NULL values for all the columns.

In [87]:
```
df.isna().sum()
```

Vishal Acharya

Vishal Acharya

Out[87]:
```
Unnamed: 0          0
track_id            0
artists             1
album_name          1
track_name          1
popularity          0
duration_ms         0
explicit            0
danceability        0
energy              0
key                 0
loudness            0
mode                0
speechiness         0
acousticness        0
instrumentalness    0
liveness            0
valence             0
tempo               0
time_signature      0
track_genre         0
dtype: int64
```

11.Remove first 3 columns from this Data Frame.

In [88]:
```
df.drop(df.columns[0:3],axis=1,inplace=True)
df.shape
```

Out[88]:
```
(114000, 18)
```

12.Remove first 10 rows from this Data Frame.

In [89]:
```
df.drop(range(0,10),axis=0,inplace=True)
df.shape
```

Out[89]:
```
(113990, 18)
```

13.After removing first 3 columns and first 10 rows from this data frame find outliers for the column popularity.

In [90]:
```
Q1 = df['popularity'].quantile(0.25)
Q3 = df['popularity'].quantile(0.75)
IQR = Q3 - Q1
# Determine the lower and upper bounds for outliers
```

Vishal Acharya

```
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
# Identify outliers
outliers = df[(df['popularity'] < lower_bound) | (df['popularity'] > upper_bound)]
# Display the outliers
print("\nOutliers in the 'popularity' column:")
print(outliers)
```

```
Outliers in the 'popularity' column:
                     album_name                track_name  popularity  \
20001  Unholy (feat. Kim Petras)  Unholy (feat. Kim Petras)         100
81051  Unholy (feat. Kim Petras)  Unholy (feat. Kim Petras)         100

       duration_ms  explicit  danceability  energy  key  loudness  mode  \
20001       156943     False         0.714   0.472    2    -7.375     1
81051       156943     False         0.714   0.472    2    -7.375     1

       speechiness  acousticness  instrumentalness  liveness  valence  \
20001       0.0864         0.013          0.000005     0.266    0.238
81051       0.0864         0.013          0.000005     0.266    0.238

         tempo  time_signature track_genre
20001  131.121               4       dance
81051  131.121               4         pop
```

14.After removing first 3 columns and first 10 rows from this data frame remove outliers for the column energy then display the data frame.

In [91]:
```
df.shape
```

Out[91]:
```
(113990, 18)
```

In [92]:
```
Q1 = df['popularity'].quantile(0.25)
Q3 = df['popularity'].quantile(0.75)
IQR = Q3 - Q1

# Determine the lower and upper bounds for outliers
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers = df[(df['popularity'] < lower_bound) | (df['popularity'] > upper_bound)]
```

```
    # Remove outliers
    df = df[~((df['popularity'] < lower_bound) | (df['popularity'] > upper_bound))]
```

In [93]:
```
df.shape
```

Out[93]: (113988, 18)

15.Display cross tabulation between time_signature and track_genre for actual Data Frame.

In [94]:
```
pd.crosstab(df.time_signature,df.track_genre)
```

Out[94]:

track_genre	acoustic	afrobeat	alt-rock	alternative	ambient	anime	black-metal	bluegrass	blues	brazil	...	spanish	study	swedish	synth po
time_signature															
0	0	0	1	0	3	0	0	0	0	0	...	0	0	0	
1	9	5	1	2	39	7	20	5	6	5	...	2	6	6	
3	99	52	56	64	264	63	129	80	97	42	...	55	49	49	1
4	869	930	940	922	648	923	838	909	894	939	...	937	934	928	98
5	13	13	2	12	46	7	13	6	3	14	...	6	11	17	

5 rows × 114 columns

Read this csv file car data.csv and display the basic information like memory and data types for this data frame.

Write Python Code for the following cases:

1. How Many ritz car are there with kms driven more than 30000km.
2. How many Petrol cars are of the year 2017 and whose selling price > 10 lakhs.
3. How many swift cars are there with selling price < 4 Lakhs.

Vishal Acharya

4. How many Automatic Transmission Petrol Car are of the year 2015 whose selling price is > 10 Lakhs.

5. List out Vehicles with Automatic Transmission and selling price < 1 Lakh.

6. How Many Petrol Vehicles are there with kms driven more than 50000kms and Year is between 2010 to 2015(both Year included).

7. List out the cars whose price difference between present price and selling price is > 15 lakhs.

8. How many Petrol vehicles are there whose kms driven < 5000km and selling price < 50000."

Read CSV and Display Basic Information

In [96]:
```
import pandas as pd

# Read CSV file
df = pd.read_csv("cardata.csv")

# Display first few rows
print(df.head())

# Display basic information (data types + memory usage)
df.info()
```

Vishal Acharya

Vishal Acharya

```
  Car_Name  Year  Selling_Price  Present_Price  Kms_Driven Fuel_Type  \
0     ritz  2014           3.35           5.59       27000    Petrol
1      sx4  2013           4.75           9.54       43000    Diesel
2     ciaz  2017           7.25           9.85        6900    Petrol
3  wagon r  2011           2.85           4.15        5200    Petrol
4    swift  2014           4.60           6.87       42450    Diesel

  Seller_Type Transmission  Owner
0      Dealer       Manual      0
1      Dealer       Manual      0
2      Dealer       Manual      0
3      Dealer       Manual      0
4      Dealer       Manual      0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 301 entries, 0 to 300
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Car_Name       301 non-null    object
 1   Year           301 non-null    int64
 2   Selling_Price  301 non-null    float64
 3   Present_Price  301 non-null    float64
 4   Kms_Driven     301 non-null    int64
 5   Fuel_Type      301 non-null    object
 6   Seller_Type    301 non-null    object
 7   Transmission   301 non-null    object
 8   Owner          301 non-null    int64
dtypes: float64(2), int64(3), object(4)
memory usage: 21.3+ KB
```

How many Ritz cars with kms driven > 30000

In [97]:
```
print(df[(df["Car_Name"] == "ritz") & (df["Kms_Driven"] > 30000)].shape[0])
```

```
3
```

How many Petrol cars of year 2017 with selling price > 10 lakhs

In [98]:
```
print(df[(df["Fuel_Type"] == "Petrol") &
         (df["Year"] == 2017) &
         (df["Selling_Price"] > 10)].shape[0])
```

2

How many Swift cars with selling price < 4 lakhs

In [99]:
```
print(df[(df["Car_Name"] == "swift") &
         (df["Selling_Price"] < 4)].shape[0])
```

1

How many Automatic Transmission Petrol cars of year 2015 with selling price > 10 lakhs

In [100…
```
print(df[(df["Fuel_Type"] == "Petrol") &
         (df["Transmission"] == "Automatic") &
         (df["Year"] == 2015) &
         (df["Selling_Price"] > 10)].shape[0])
```

1

List vehicles with Automatic Transmission and selling price < 1 lakh

In [101…
```
print(df[(df["Transmission"] == "Automatic") &
         (df["Selling_Price"] < 1)])
```

Vishal Acharya

Vishal Acharya

```
              Car_Name  Year  Selling_Price  Present_Price  Kms_Driven  \
155    Honda Activa 4G  2017           0.48           0.51        4300
159    Honda Activa 4G  2017           0.45           0.51        4000
165          Activa 3g  2016           0.45           0.54         500
173          Activa 4g  2017           0.40           0.51        1300
177   Honda Activa 125  2016           0.35           0.57       24000
178        TVS Jupyter  2014           0.35           0.52       19000
185  Suzuki Access 125  2008           0.25           0.58        1900
186           TVS Wego  2010           0.25           0.52       22000
196          Activa 3g  2008           0.17           0.52      500000

    Fuel_Type Seller_Type Transmission  Owner
155    Petrol  Individual    Automatic      0
159    Petrol  Individual    Automatic      0
165    Petrol  Individual    Automatic      0
173    Petrol  Individual    Automatic      0
177    Petrol  Individual    Automatic      0
178    Petrol  Individual    Automatic      0
185    Petrol  Individual    Automatic      0
186    Petrol  Individual    Automatic      0
196    Petrol  Individual    Automatic      0
```

How many Petrol vehicles with kms driven > 50000 and year between 2010–2015

In [102…

```
print(df[(df["Fuel_Type"] == "Petrol") &
         (df["Kms_Driven"] > 50000) &
         (df["Year"] >= 2010) &
         (df["Year"] <= 2015)].shape[0])
```

```
23
```

List cars where Present Price − Selling Price > 15 lakhs

In [103…

```
print(df[(df["Present_Price"] - df["Selling_Price"]) > 15])
```

Vishal Acharya

```
         Car_Name  Year  Selling_Price  Present_Price  Kms_Driven Fuel_Type  \
50       fortuner  2012          14.90          30.61      104707    Diesel
59       fortuner  2014          19.99          35.96       41000    Diesel
62       fortuner  2014          18.75          35.96       78000    Diesel
78  corolla altis  2010           5.25          22.83       80000    Petrol
79       fortuner  2012          14.50          30.61       89000    Diesel
85          camry  2006           2.50          23.73      142000    Petrol
86   land cruiser  2010          35.00          92.60       78000    Diesel
94  corolla altis  2008           4.00          22.78       89000    Petrol

   Seller_Type Transmission  Owner
50      Dealer    Automatic      0
59      Dealer    Automatic      0
62      Dealer    Automatic      0
78      Dealer    Automatic      0
79      Dealer    Automatic      0
85  Individual    Automatic      3
86      Dealer       Manual      0
94      Dealer    Automatic      0
```

How many Petrol vehicles with kms driven < 5000 and selling price < 50000

In [104…

```
print(df[(df["Fuel_Type"] == "Petrol") &
         (df["Kms_Driven"] < 5000) &
         (df["Selling_Price"] < 50000)].shape[0])
```

```
21
```

Most counting questions can be written as:

df[condition].shape[0]

len(df[condition])

65 "1. Load the dataset into a pandas DataFrame (data_result.csv) and answer the following questions.

2. View the first few rows of the dataset
3. Check the shape of the dataset

Vishal Acharya

4. View the first last rows of the dataset
5. Get summary statistics of numerical columns
6. Get summary statistics of numerical columns with 0.58 and 0.87 percentiles
7. Get summary statistics of all types of columns
8. Information of all columns
9. Check for missing values
10. Removing duplicates if duplicates
11. List out female students who have greater than 7 spi in all semesters.
12. Find number of students those who have greater than 8 spi in all 5 semesters.
13. Find outliers of sem 4 result. Also represent statistical analysis with visualization.(boxplot)"

Load the dataset into a pandas DataFrame

In [105…

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("data_result.csv")
```

View the first few rows of the dataset

In [106…

```
print(df.head())
```

```
   Student_ID       Name  Gender  Sem1_SPI  Sem2_SPI  Sem3_SPI  Sem4_SPI  \
0        1001  Student_1    Male      6.56      6.31      4.77      8.70
1        1002  Student_2  Female      5.94      5.10      7.32      8.68
2        1003  Student_3    Male      8.89      9.40      7.37      4.98
3        1004  Student_4    Male      6.39      9.15      7.88      7.12
4        1005  Student_5    Male      5.99      5.87      8.35      4.81

   Sem5_SPI
0      9.56
1      7.71
2      5.71
3      8.06
4      7.78
```

Check the shape of the dataset

In [107…
```
print(df.shape)
```

```
(122, 8)
```

View the first and last rows of the dataset

In [108…
```
print(df.head())
print(df.tail())
```

```
   Student_ID       Name  Gender  Sem1_SPI  Sem2_SPI  Sem3_SPI  Sem4_SPI  \
0        1001  Student_1    Male      6.56      6.31      4.77      8.70
1        1002  Student_2  Female      5.94      5.10      7.32      8.68
2        1003  Student_3    Male      8.89      9.40      7.37      4.98
3        1004  Student_4    Male      6.39      9.15      7.88      7.12
4        1005  Student_5    Male      5.99      5.87      8.35      4.81

   Sem5_SPI
0      9.56
1      7.71
2      5.71
3      8.06
4      7.78
     Student_ID         Name  Gender  Sem1_SPI  Sem2_SPI  Sem3_SPI  Sem4_SPI  \
117        1118  Student_118  Female      6.55      5.64      6.24      5.81
118        1119  Student_119  Female      9.46      7.80      8.06      6.89
119        1120  Student_120    Male      5.23      4.95      8.49      5.18
120        1011   Student_11    Male      8.59      4.99      6.83      5.26
121        1026   Student_26  Female      6.22      4.55      8.41      9.52

     Sem5_SPI
117      6.42
118      7.92
119      7.53
120      7.26
121      7.63
```

Get summary statistics of numerical columns

Vishal Acharya

In [109…

```
print(df.describe())
```

```
       Student_ID    Sem1_SPI    Sem2_SPI    Sem3_SPI    Sem4_SPI    Sem5_SPI
count   122.000000  122.000000  121.000000  122.000000  121.000000  122.000000
mean   1059.811475    7.020820    7.261488    7.157295    7.372397    6.882213
std      34.922898    1.529487    1.577154    1.555221    1.711257    1.413952
min    1001.000000    4.530000    4.530000    4.560000    4.560000    4.580000
25%    1029.250000    5.762500    5.910000    5.982500    5.950000    5.612500
50%    1059.500000    7.045000    7.450000    7.210000    7.460000    6.935000
75%    1089.750000    8.367500    8.710000    8.462500    8.940000    7.890000
max    1120.000000    9.730000    9.750000    9.750000    9.800000    9.690000
```

Get summary statistics with 0.58 and 0.87 percentiles

In [110…

```
print(df.describe(percentiles=[0.58, 0.87]))
```

```
       Student_ID    Sem1_SPI    Sem2_SPI    Sem3_SPI    Sem4_SPI    Sem5_SPI
count   122.000000  122.000000  121.000000  122.000000  121.000000  122.000000
mean   1059.811475    7.020820    7.261488    7.157295    7.372397    6.882213
std      34.922898    1.529487    1.577154    1.555221    1.711257    1.413952
min    1001.000000    4.530000    4.530000    4.560000    4.560000    4.580000
50%    1059.500000    7.045000    7.450000    7.210000    7.460000    6.935000
58%    1069.180000    7.343600    7.856000    7.743600    7.900000    7.227200
87%    1104.270000    8.900800    9.224000    9.042700    9.520000    8.658100
max    1120.000000    9.730000    9.750000    9.750000    9.800000    9.690000
```

Get summary statistics of all types of columns

In [111…

```
print(df.describe(include="all"))
```

Vishal Acharya

Vishal Acharya

```
         Student_ID        Name  Gender    Sem1_SPI    Sem2_SPI    Sem3_SPI  \
count    122.000000         122     122  122.000000  121.000000  122.000000
unique          NaN         120       2         NaN         NaN         NaN
top             NaN  Student_26  Female         NaN         NaN         NaN
freq            NaN           2      71         NaN         NaN         NaN
mean    1059.811475         NaN     NaN    7.020820    7.261488    7.157295
std       34.922898         NaN     NaN    1.529487    1.577154    1.555221
min     1001.000000         NaN     NaN    4.530000    4.530000    4.560000
25%     1029.250000         NaN     NaN    5.762500    5.910000    5.982500
50%     1059.500000         NaN     NaN    7.045000    7.450000    7.210000
75%     1089.750000         NaN     NaN    8.367500    8.710000    8.462500
max     1120.000000         NaN     NaN    9.730000    9.750000    9.750000

          Sem4_SPI    Sem5_SPI
count   121.000000  122.000000
unique         NaN         NaN
top            NaN         NaN
freq           NaN         NaN
mean      7.372397    6.882213
std       1.711257    1.413952
min       4.560000    4.580000
25%       5.950000    5.612500
50%       7.460000    6.935000
75%       8.940000    7.890000
max       9.800000    9.690000
```

Information of all columns

In [112…

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 122 entries, 0 to 121
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Student_ID  122 non-null    int64
 1   Name        122 non-null    object
 2   Gender      122 non-null    object
 3   Sem1_SPI    122 non-null    float64
 4   Sem2_SPI    121 non-null    float64
 5   Sem3_SPI    122 non-null    float64
 6   Sem4_SPI    121 non-null    float64
 7   Sem5_SPI    122 non-null    float64
dtypes: float64(5), int64(1), object(2)
memory usage: 7.8+ KB
None
```

Check for missing values

In [114…

```
print(df.isnull().sum())
```

```
Student_ID    0
Name          0
Gender        0
Sem1_SPI      0
Sem2_SPI      1
Sem3_SPI      0
Sem4_SPI      1
Sem5_SPI      0
dtype: int64
```

Remove duplicate rows (if any)

In [115…

```
df = df.drop_duplicates()
print(df.shape)
```

```
(120, 8)
```

ist female students with SPI > 7 in all semesters

Vishal Acharya

Vishal Acharya

In [116…

```
result1 = df[
    (df["Gender"] == "Female") &
    (df["Sem1_SPI"] > 7) &
    (df["Sem2_SPI"] > 7) &
    (df["Sem3_SPI"] > 7) &
    (df["Sem4_SPI"] > 7) &
    (df["Sem5_SPI"] > 7)
]

print(result1)
```

```
     Student_ID         Name  Gender  Sem1_SPI  Sem2_SPI  Sem3_SPI  Sem4_SPI  \
55         1056   Student_56  Female      9.12      8.33      7.90      9.76
90         1091   Student_91  Female      9.31      8.79      9.75      7.75
93         1094   Student_94  Female      7.09      9.34      8.61      9.09
112        1113  Student_113  Female      7.21      8.86      9.54      9.78

     Sem5_SPI
55       7.01
90       8.28
93       7.32
112      7.61
```

Number of students with SPI > 8 in all 5 semesters

In [117…

```
result2 = df[
    (df["Sem1_SPI"] > 8) &
    (df["Sem2_SPI"] > 8) &
    (df["Sem3_SPI"] > 8) &
    (df["Sem4_SPI"] > 8) &
    (df["Sem5_SPI"] > 8)
]

print(result2.shape[0])
```

```
0
```

Find outliers in Sem4 result (IQR method)

Outlier Detection

In [118…

```
Q1 = df["Sem4_SPI"].quantile(0.25)
Q3 = df["Sem4_SPI"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df["Sem4_SPI"] < lower) | (df["Sem4_SPI"] > upper)]

print(outliers)
```

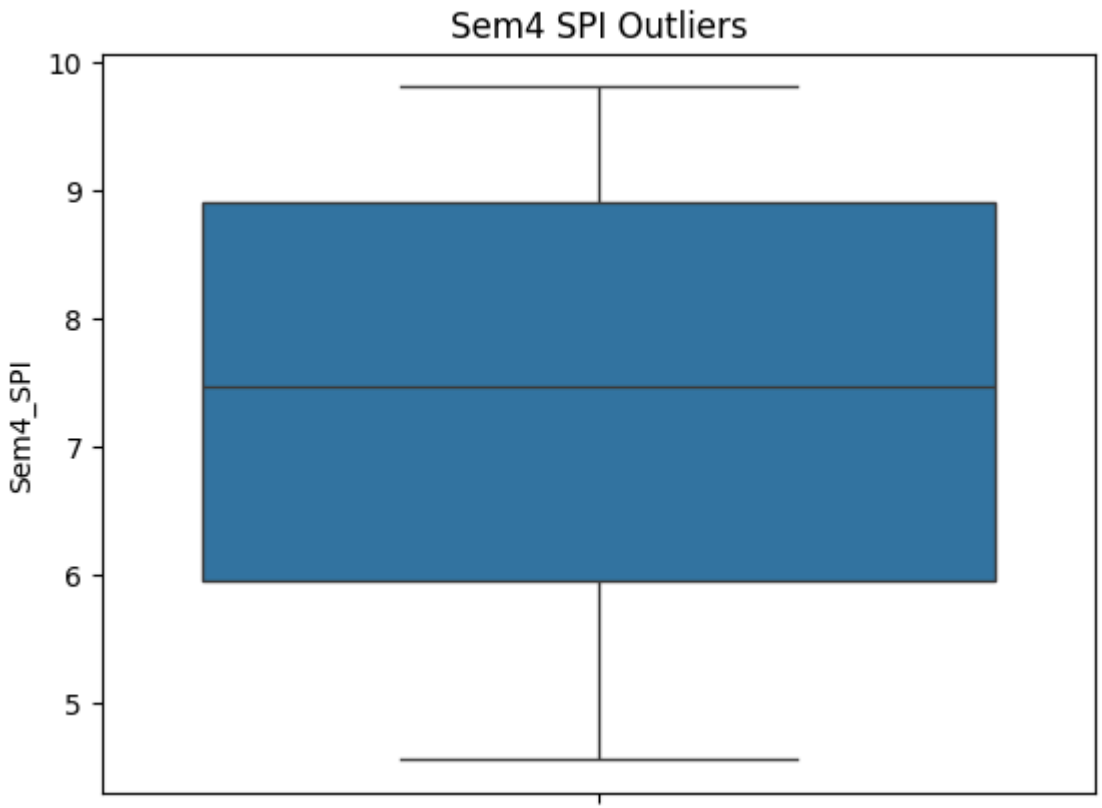
```
Empty DataFrame
Columns: [Student_ID, Name, Gender, Sem1_SPI, Sem2_SPI, Sem3_SPI, Sem4_SPI, Sem5_SPI]
Index: []
```

In [119…

```
sns.boxplot(y=df["Sem4_SPI"])
plt.title("Sem4 SPI Outliers")
plt.show()
```

Vishal Acharya

Vishal Acharya



66 "Use the file movies.csv which contains 1629 rows and 18 columns. Read this csv file and display the basic information like memory and data types for this data frame.

Write python code for the following cases:

1. List out Movies Released in Year 2019.
2. How Many Movies are having IMDB Rating > 7 (Display Number of Movies).
3. List out the Movies with ‘title’ and ‘story’ whose IMDB Votes > 20000.
4. List out Movies Released in Year 2018, Display only Movie Title with Release Date of Year 2018 Movies.

5. Display only Movie Title with its Wikipedia Link."

In [121…

```
import pandas as pd

# Load dataset
df = pd.read_csv("movies.csv")

# View first rows
print(df.head())

# Display dataset information (data types + memory usage)
df.info()

# Column-wise memory usage
print(df.memory_usage())
```

Vishal Acharya

Vishal Acharya

```
                                title_x    imdb_id  \
0             Uri: The Surgical Strike  tt8291224
1                         Battalion 609  tt9472208
2  The Accidental Prime Minister (film)  tt6986710
3                       Why Cheat India  tt8108208
4                       Evening Shadows  tt6028796

                                        poster_path  \
0  https://upload.wikimedia.org/wikipedia/en/thum...
1                                                NaN
2  https://upload.wikimedia.org/wikipedia/en/thum...
3  https://upload.wikimedia.org/wikipedia/en/thum...
4                                                NaN

                                          wiki_link  \
0  https://en.wikipedia.org/wiki/Uri:_The_Surgica...
1       https://en.wikipedia.org/wiki/Battalion_609
2  https://en.wikipedia.org/wiki/The_Accidental_P...
3     https://en.wikipedia.org/wiki/Why_Cheat_India
4     https://en.wikipedia.org/wiki/Evening_Shadows

                        title_y                original_title  is_adult  \
0      Uri: The Surgical Strike       Uri: The Surgical Strike         0
1                 Battalion 609                 Battalion 609         0
2  The Accidental Prime Minister  The Accidental Prime Minister         0
3               Why Cheat India                Why Cheat India         0
4               Evening Shadows                Evening Shadows         0

   year_of_release runtime            genres  imdb_rating  imdb_votes  \
0             2019     138  Action|Drama|War          8.4       35112
1             2019     131               War          4.1          73
2             2019     112   Biography|Drama          6.1        5549
3             2019     121       Crime|Drama          6.0        1891
4             2018     102             Drama          7.3         280

                                              story  \
0  Divided over five chapters  the film chronicle...
1  The story revolves around a cricket match betw...
2  Based on the memoir by Indian policy analyst S...
3  The movie focuses on existing malpractices in ...
4  While gay rights and marriage equality has bee...
```

Vishal Acharya

```
                                                summary tagline  \
0  Indian army special forces execute a covert op...     NaN
1  The story of Battalion 609 revolves around a c...     NaN
2  Explores Manmohan Singh's tenure as the Prime ...     NaN
3  The movie focuses on existing malpractices in ...     NaN
4  Under the 'Evening Shadows'  truth often plays...     NaN

                                                 actors        wins_nominations  \
0  Vicky Kaushal|Paresh Rawal|Mohit Raina|Yami Ga...                   4 wins
1  Vicky Ahuja|Shoaib Ibrahim|Shrikant Kamat|Elen...                      NaN
2  Anupam Kher|Akshaye Khanna|Aahana Kumra|Atul S...                      NaN
3  Emraan Hashmi|Shreya Dhanwanthary|Snighdadeep ...                      NaN
4  Mona Ambegaonkar|Ananth Narayan Mahadevan|Deva...  17 wins & 1 nomination

              release_date
0    11 January 2019 (USA)
1  11 January 2019 (India)
2    11 January 2019 (USA)
3    18 January 2019 (USA)
4  11 January 2019 (India)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1629 entries, 0 to 1628
Data columns (total 18 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   title_x           1629 non-null   object
 1   imdb_id           1629 non-null   object
 2   poster_path       1526 non-null   object
 3   wiki_link         1629 non-null   object
 4   title_y           1629 non-null   object
 5   original_title    1629 non-null   object
 6   is_adult          1629 non-null   int64
 7   year_of_release   1629 non-null   int64
 8   runtime           1629 non-null   object
 9   genres            1629 non-null   object
 10  imdb_rating       1629 non-null   float64
 11  imdb_votes        1629 non-null   int64
 12  story             1609 non-null   object
 13  summary           1629 non-null   object
 14  tagline           557 non-null    object
```

Vishal Acharya

```
 15  actors            1624 non-null   object
 16  wins_nominations  707 non-null    object
 17  release_date      1522 non-null   object
dtypes: float64(1), int64(3), object(14)
memory usage: 229.2+ KB
Index                 132
title_x             13032
imdb_id             13032
poster_path         13032
wiki_link           13032
title_y             13032
original_title      13032
is_adult            13032
year_of_release     13032
runtime             13032
genres              13032
imdb_rating         13032
imdb_votes          13032
story               13032
summary             13032
tagline             13032
actors              13032
wins_nominations    13032
release_date        13032
dtype: int64
```

In [125…

```
df.columns
```

Out[125…

```
Index(['title_x', 'imdb_id', 'poster_path', 'wiki_link', 'title_y',
       'original_title', 'is_adult', 'year_of_release', 'runtime', 'genres',
       'imdb_rating', 'imdb_votes', 'story', 'summary', 'tagline', 'actors',
       'wins_nominations', 'release_date'],
      dtype='object')
```

List Movies Released in Year 2019

In [126…

```
movies_2019 = df[df["year_of_release"] == 2019]
print(movies_2019)
```

Vishal Acharya

```
                                  title_x     imdb_id  \
0                 Uri: The Surgical Strike   tt8291224
1                            Battalion 609   tt9472208
2     The Accidental Prime Minister (film)   tt6986710
3                         Why Cheat India   tt8108208
6                           Fraud Saiyaan   tt5013008
...                                   ...         ...
76                       Commando 3 (film)   tt8983168
77                              Mardaani 2   tt5668770
78                               Dabangg 3   tt7059844
79                              Good Newwz   tt8504014
1627                                 Daaka  tt10833860

                                          poster_path  \
0     https://upload.wikimedia.org/wikipedia/en/thum...
1                                                   NaN
2     https://upload.wikimedia.org/wikipedia/en/thum...
3     https://upload.wikimedia.org/wikipedia/en/thum...
6     https://upload.wikimedia.org/wikipedia/en/thum...
...                                                 ...
76    https://upload.wikimedia.org/wikipedia/en/thum...
77    https://upload.wikimedia.org/wikipedia/en/thum...
78    https://upload.wikimedia.org/wikipedia/en/thum...
79                                                  NaN
1627  https://upload.wikimedia.org/wikipedia/en/thum...

                                           wiki_link  \
0     https://en.wikipedia.org/wiki/Uri:_The_Surgica...
1           https://en.wikipedia.org/wiki/Battalion_609
2     https://en.wikipedia.org/wiki/The_Accidental_P...
3         https://en.wikipedia.org/wiki/Why_Cheat_India
6           https://en.wikipedia.org/wiki/Fraud_Saiyaan
...                                                 ...
76      https://en.wikipedia.org/wiki/Commando_3_(film)
77             https://en.wikipedia.org/wiki/Mardaani_2
78              https://en.wikipedia.org/wiki/Dabangg_3
79             https://en.wikipedia.org/wiki/Good_Newwz
1627                https://en.wikipedia.org/wiki/Daaka

                           title_y                original_title  is_adult  \
0          Uri: The Surgical Strike       Uri: The Surgical Strike         0
```

Vishal Acharya

```
1                    Battalion 609                  Battalion 609         0
2     The Accidental Prime Minister  The Accidental Prime Minister         0
3                  Why Cheat India                Why Cheat India         0
6                    Fraud Saiyaan                  Fraud Saiyyan         0
...                            ...                            ...       ...
76                      Commando 3                     Commando 3         0
77                      Mardaani 2                     Mardaani 2         0
78                       Dabangg 3                      Dabangg 3         0
79                      Good Newwz                     Good Newwz         0
1627                         Daaka                          Daaka         0

      year_of_release runtime                genres  imdb_rating  imdb_votes  \
0                2019     138       Action|Drama|War          8.4       35112
1                2019     131                   War          4.1          73
2                2019     112        Biography|Drama          6.1        5549
3                2019     121            Crime|Drama          6.0        1891
6                2019     109           Comedy|Drama          4.2         504
...               ...     ...                    ...          ...         ...
76               2019     133        Action|Thriller          5.7        2400
77               2019     103  Action|Drama|Thriller          7.4        8100
78               2019     160                 Action          3.0       15000
79               2019     134   Comedy|Drama|Romance          6.9       16000
1627             2019     136                 Action          7.4          38

                                                 story  \
0     Divided over five chapters  the film chronicle...
1     The story revolves around a cricket match betw...
2     Based on the memoir by Indian policy analyst S...
3     The movie focuses on existing malpractices in ...
6     Fraud Saiyyan is the story of a con artist in ...
...                                                 ...
76    A mysterious man is on an impending mission to...
77    Shivani Shivaji Roy locks horns with the devil...
78        Third installment of the Dabanng film series.
79    It is an upcoming Indian romantic comedy film ...
1627  Shinda tries robbing a bank so he can be wealt...

                                              summary tagline  \
0     Indian army special forces execute a covert op...     NaN
1     The story of Battalion 609 revolves around a c...     NaN
2     Explores Manmohan Singh's tenure as the Prime ...     NaN
```

Vishal Acharya

```
3     The movie focuses on existing malpractices in ...     NaN
6     Fraud Saiyyan is the story of a con artist in ...     NaN
...                                                  ...     ...
76    Commando 3 is a Hindi movie starring Vidyut Ja...     NaN
77    Shivani Shivaji Roy locks horns with the devil...     NaN
78        Third installment of the Dabanng film series.     NaN
79    It is an upcoming Indian romantic comedy film ...     NaN
1627  Shinda tries robbing a bank so he can be wealt...     NaN

                                                actors wins_nominations  \
0     Vicky Kaushal|Paresh Rawal|Mohit Raina|Yami Ga...           4 wins
1     Vicky Ahuja|Shoaib Ibrahim|Shrikant Kamat|Elen...              NaN
2     Anupam Kher|Akshaye Khanna|Aahana Kumra|Atul S...              NaN
3     Emraan Hashmi|Shreya Dhanwanthary|Snighdadeep ...              NaN
6     Arshad Warsi|Saurabh Shukla|Flora Saini|Sara L...              NaN
...                                                  ...              ...
76    Vidyut Jammwal|Adah Sharma|Angira Dhar|Sumeet ...              NaN
77             Rani Mukerji|Rajesh Sharma|Shruti Bapna|              NaN
78    Salman Khan|Sonakshi Sinha|Sudeep|Mahesh Manjr...              NaN
79    Akshay Kumar|Kareena Kapoor|Kiara Advani|Tisca...              NaN
1627                         Gippy Grewal|Zareen Khan|              NaN

                 release_date
0       11 January 2019 (USA)
1     11 January 2019 (India)
2       11 January 2019 (USA)
3       18 January 2019 (USA)
6     18 January 2019 (India)
...                       ...
76                        NaN
77                        NaN
78                        NaN
79                        NaN
1627    1 November 2019 (USA)

[75 rows x 18 columns]
```

How Many Movies have IMDB Rating > 7

In [127…

```
count_movies = df[df["imdb_rating"] > 7].shape[0]
print(count_movies)
```

```
331
```

In [128…

```
len(df[df["imdb_rating"] > 7])
```

Out[128…

```
331
```

List Movies with title and story whose IMDB Votes > 20000

In [130…

```
result = df[df["imdb_votes"] > 20000][["title_x","story"]]
print(result)
```

Vishal Acharya

Vishal Acharya

```
                          title_x  \
0        Uri: The Surgical Strike
11                      Gully Boy
36                    Kabir Singh
74                    Dil Bechara
88                      Padmaavat
...                           ...
1490     Devdas (2002 Hindi film)
1565  Kabhi Khushi Kabhie Gham...
1567                       Lagaan
1568                       Lagaan
1571               Dil Chahta Hai

                                                  story
0     Divided over five chapters  the film chronicle...
11    Gully Boy is a film about a 22-year-old boy "M...
36    This Sandeep Vanga directorial is a remake of ...
74              A love story about two cancer patients.
88    This fictional story is set in 13th century me...
...                                                 ...
1490  Devdas Mukherji is black-listed by his multi-m...
1565  Yashvardhan Raichand lives a very wealthy life...
1567  This is the story about the resilience shown b...
1568  This is the story about the resilience shown b...
1571  Three young men Akash  Sameer  and Siddharth a...

[105 rows x 2 columns]
```

List Movies Released in Year 2018

Display only Movie Title and Release Date

In [132…

```
movies_2018 = df[df["year_of_release"] == 2018][["title_x","release_date"]]
print(movies_2018)
```

```
                        title_x              release_date
4               Evening Shadows   11 January 2019 (India)
5                   Soni (film)     18 January 2019 (USA)
16       Mard Ko Dard Nahi Hota       22 March 2019 (USA)
17                 Hamid (film)     15 March 2019 (India)
20    Mere Pyare Prime Minister     15 March 2019 (India)
...                         ...                       ...
156                Rajma Chawal  30 November 2018 (India)
157            Zero (2018 film)    21 December 2018 (USA)
158                      Simmba    28 December 2018 (USA)
166          Thugs of Hindostan     8 November 2018 (USA)
1626            Sabse Bada Sukh                       NaN

[79 rows x 2 columns]
```

In [ ]:
```
Display only Movie Title with Wikipedia Link
```

In [134…
```
wiki_movies = df[["title_x","wiki_link"]]
print(wiki_movies)
```

Vishal Acharya

Vishal Acharya

```
                                 title_x  \
0                Uri: The Surgical Strike
1                           Battalion 609
2     The Accidental Prime Minister (film)
3                        Why Cheat India
4                        Evening Shadows
...                                  ...
1624                Tera Mera Saath Rahen
1625                 Yeh Zindagi Ka Safar
1626                      Sabse Bada Sukh
1627                                Daaka
1628                             Humsafar

                                          wiki_link
0     https://en.wikipedia.org/wiki/Uri:_The_Surgica...
1           https://en.wikipedia.org/wiki/Battalion_609
2     https://en.wikipedia.org/wiki/The_Accidental_P...
3         https://en.wikipedia.org/wiki/Why_Cheat_India
4         https://en.wikipedia.org/wiki/Evening_Shadows
...                                                 ...
1624  https://en.wikipedia.org/wiki/Tera_Mera_Saath_...
1625  https://en.wikipedia.org/wiki/Yeh_Zindagi_Ka_S...
1626      https://en.wikipedia.org/wiki/Sabse_Bada_Sukh
1627                https://en.wikipedia.org/wiki/Daaka
1628             https://en.wikipedia.org/wiki/Humsafar

[1629 rows x 2 columns]
```

67 "You are given a CSV file named ipl-matches.csv, which contains data of all Indian Premier League (IPL) matches played from the year 2008 to 2022.Read the dataset and display the basic information about the DataFrame.

Using Python and Pandas, write code to answer the following questions:

1. How many matches were won by Gujarat Titans in a Super Over ?

2. How Many Matches won by Royal Challengers Bangalore Vs Kolkata Knight Riders.

3. How many Matches won by Toss Winner of the Match.

Vishal Acharya

4. In how many matches was the result decided by the D/L method and the match was won by wickets?

5. In how many matches Toss won by Rajasthan Royals & Won the Match.

6. In how many matches V Kohli declared player of the match against Chennai Super Kings.

7. Which Team is the Toss Winner of the Final match Played in Dubai?

8. List out the Finals played by Kolkata Knight Riders in which Toss Won by Kolkata Knight Riders in the Finals and choose to field."

Read dataset and display basic information

In [137…

```
import pandas as pd

# Load dataset
df = pd.read_csv("ipl-matches.csv")

# View first rows
print(df.head())

# Basic information (data types + memory)
df.info()

# Shape of dataset
print(df.shape)
df.columns
```

Vishal Acharya

```
        ID       City        Date Season  MatchNumber  \
0  1312200  Ahmedabad  2022-05-29   2022        Final
1  1312199  Ahmedabad  2022-05-27   2022  Qualifier 2
2  1312198    Kolkata  2022-05-25   2022   Eliminator
3  1312197    Kolkata  2022-05-24   2022  Qualifier 1
4  1304116     Mumbai  2022-05-22   2022           70

                       Team1                 Team2  \
0            Rajasthan Royals        Gujarat Titans
1  Royal Challengers Bangalore      Rajasthan Royals
2  Royal Challengers Bangalore  Lucknow Super Giants
3            Rajasthan Royals        Gujarat Titans
4         Sunrisers Hyderabad          Punjab Kings

                            Venue            TossWinner TossDecision  \
0  Narendra Modi Stadium, Ahmedabad      Rajasthan Royals          bat
1  Narendra Modi Stadium, Ahmedabad      Rajasthan Royals        field
2            Eden Gardens, Kolkata  Lucknow Super Giants        field
3            Eden Gardens, Kolkata        Gujarat Titans        field
4         Wankhede Stadium, Mumbai   Sunrisers Hyderabad          bat

  SuperOver                  WinningTeam    WonBy  Margin method  \
0         N               Gujarat Titans  Wickets     7.0    NaN
1         N             Rajasthan Royals  Wickets     7.0    NaN
2         N  Royal Challengers Bangalore     Runs    14.0    NaN
3         N               Gujarat Titans  Wickets     7.0    NaN
4         N                 Punjab Kings  Wickets     5.0    NaN

  Player_of_Match                                       Team1Players  \
0      HH Pandya  ['YBK Jaiswal', 'JC Buttler', 'SV Samson', 'D ...
1     JC Buttler  ['V Kohli', 'F du Plessis', 'RM Patidar', 'GJ ...
2     RM Patidar  ['V Kohli', 'F du Plessis', 'RM Patidar', 'GJ ...
3      DA Miller  ['YBK Jaiswal', 'JC Buttler', 'SV Samson', 'D ...
4   Harpreet Brar  ['PK Garg', 'Abhishek Sharma', 'RA Tripathi', ...

                                       Team2Players        Umpire1  \
0  ['WP Saha', 'Shubman Gill', 'MS Wade', 'HH Pan...    CB Gaffaney
1  ['YBK Jaiswal', 'JC Buttler', 'SV Samson', 'D ...    CB Gaffaney
2  ['Q de Kock', 'KL Rahul', 'M Vohra', 'DJ Hooda...  J Madanagopal
3  ['WP Saha', 'Shubman Gill', 'MS Wade', 'HH Pan...   BNJ Oxenford
4  ['JM Bairstow', 'S Dhawan', 'M Shahrukh Khan',...   AK Chaudhary
```

Vishal Acharya

```
         Umpire2
0    Nitin Menon
1    Nitin Menon
2       MA Gough
3      VK Sharma
4  NA Patwardhan
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 950 entries, 0 to 949
Data columns (total 20 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   ID               950 non-null    int64
 1   City             899 non-null    object
 2   Date             950 non-null    object
 3   Season           950 non-null    object
 4   MatchNumber      950 non-null    object
 5   Team1            950 non-null    object
 6   Team2            950 non-null    object
 7   Venue            950 non-null    object
 8   TossWinner       950 non-null    object
 9   TossDecision     950 non-null    object
 10  SuperOver        946 non-null    object
 11  WinningTeam      946 non-null    object
 12  WonBy            950 non-null    object
 13  Margin           932 non-null    float64
 14  method           19 non-null     object
 15  Player_of_Match  946 non-null    object
 16  Team1Players     950 non-null    object
 17  Team2Players     950 non-null    object
 18  Umpire1          950 non-null    object
 19  Umpire2          950 non-null    object
dtypes: float64(1), int64(1), object(18)
memory usage: 148.6+ KB
(950, 20)
```

Out[137…

```
Index(['ID', 'City', 'Date', 'Season', 'MatchNumber', 'Team1', 'Team2',
       'Venue', 'TossWinner', 'TossDecision', 'SuperOver', 'WinningTeam',
       'WonBy', 'Margin', 'method', 'Player_of_Match', 'Team1Players',
       'Team2Players', 'Umpire1', 'Umpire2'],
      dtype='object')
```

Matches won by Gujarat Titans in a Super Over

In [138…
```
q1 = df[(df["WinningTeam"] == "Gujarat Titans") & (df["SuperOver"] == "Y")]
print(q1.shape[0])
```

```
0
```

Matches won by Royal Challengers Bangalore vs Kolkata Knight Riders

In [139…
```
q2 = df[
    ((df["Team1"] == "Royal Challengers Bangalore") & (df["Team2"] == "Kolkata Knight Riders")) |
    ((df["Team1"] == "Kolkata Knight Riders") & (df["Team2"] == "Royal Challengers Bangalore"))
]

print(q2["WinningTeam"].value_counts())
```

```
WinningTeam
Kolkata Knight Riders          16
Royal Challengers Bangalore    14
Name: count, dtype: int64
```

Matches where Toss Winner also Won the Match

In [140…
```
q3 = df[df["TossWinner"] == df["WinningTeam"]]
print(q3.shape[0])
```

```
489
```

Matches decided by D/L method and won by wickets

In [141…
```
q4 = df[(df["method"] == "D/L") & (df["WonBy"] == "Wickets")]
print(q4.shape[0])
```

```
8
```

Matches where Rajasthan Royals won the toss and also won the match

In [142…
```
q5 = df[(df["TossWinner"] == "Rajasthan Royals") & (df["WinningTeam"] == "Rajasthan Royals")]
print(q5.shape[0])
```

Vishal Acharya

```
49
```

Matches where V Kohli was Player of the Match against Chennai Super Kings

In [143…

```
q6 = df[
    (df["Player_of_Match"] == "V Kohli") &
    ((df["Team1"] == "Chennai Super Kings") | (df["Team2"] == "Chennai Super Kings"))
]

print(q6.shape[0])
```

```
2
```

Toss Winner of the Final match played in Dubai

In [144…

```
q7 = df[(df["MatchNumber"] == "Final") & (df["City"] == "Dubai")]
print(q7[["Season","TossWinner"]])
```

```
   Season             TossWinner
74   2021  Kolkata Knight Riders
```

Finals played by Kolkata Knight Riders where KKR won toss and chose to field

In [145…

```
q8 = df[
    (df["MatchNumber"] == "Final") &
    ((df["Team1"] == "Kolkata Knight Riders") | (df["Team2"] == "Kolkata Knight Riders")) &
    (df["TossWinner"] == "Kolkata Knight Riders") &
    (df["TossDecision"] == "field")
]

print(q8)
```

Vishal Acharya

Vishal Acharya

```
          ID       City        Date Season MatchNumber                  Team1  \
74   1254117      Dubai  2021-10-15   2021       Final    Chennai Super Kings
492   734049  Bangalore  2014-06-01   2014       Final  Kolkata Knight Riders

                    Team2                                Venue  \
74   Kolkata Knight Riders  Dubai International Cricket Stadium
492        Kings XI Punjab                M Chinnaswamy Stadium

               TossWinner TossDecision SuperOver            WinningTeam  \
74   Kolkata Knight Riders        field         N    Chennai Super Kings
492  Kolkata Knight Riders        field         N  Kolkata Knight Riders

       WonBy  Margin method Player_of_Match  \
74      Runs    27.0    NaN    F du Plessis
492  Wickets     3.0    NaN       MK Pandey

                                       Team1Players  \
74   ['RD Gaikwad', 'F du Plessis', 'RV Uthappa', '...
492  ['RV Uthappa', 'G Gambhir', 'MK Pandey', 'YK P...

                                       Team2Players          Umpire1  \
74   ['Shubman Gill', 'VR Iyer', 'N Rana', 'SP Nari...      Nitin Menon
492  ['V Sehwag', 'M Vohra', 'GJ Bailey', 'WP Saha'...  HDPK Dharmasena

           Umpire2
74   RK Illingworth
492    BNJ Oxenford
```

68 "import pandas as pd

data = { ""EmpID"":[101,102,103,104,105], ""Dept"":[""IT"",""HR"",""IT"",""Finance"",""HR""], ""Salary"":[60000,45000,80000,75000,50000], ""Age"":[25,32,29,41,28] }

df = pd.DataFrame(data) Sort employees by Dept (Ascending) and Salary (Descending). After sorting, sort the index in descending order. Display the final DataFrame."

In [146…

```
import pandas as pd

data = {
```

Vishal Acharya

```
    "EmpID":[101,102,103,104,105],
    "Dept":["IT","HR","IT","Finance","HR"],
    "Salary":[60000,45000,80000,75000,50000],
    "Age":[25,32,29,41,28]
}

df = pd.DataFrame(data)

# Sort by Department (Ascending) and Salary (Descending)
df_sorted = df.sort_values(by=["Dept","Salary"], ascending=[True, False])

print(df_sorted)

# Sort index in descending order
df_final = df_sorted.sort_index(ascending=False)

# Display final DataFrame
print(df_final)
```

```
   EmpID     Dept  Salary  Age
3    104  Finance   75000   41
4    105       HR   50000   28
1    102       HR   45000   32
2    103       IT   80000   29
0    101       IT   60000   25
   EmpID     Dept  Salary  Age
4    105       HR   50000   28
3    104  Finance   75000   41
2    103       IT   80000   29
1    102       HR   45000   32
0    101       IT   60000   25
```

69 "import panda as pd

df = pd.DataFrame({ ""Student"":[""A"",""B"",""C"",""D""], ""Math"":[70,45,90,60], ""Science"":[75,40,85,55] }) Create a column Average using apply() row-wise. Create a column Grade based on: ≥ 80 → Distinction ≥ 60 → First Otherwise → Second Display final DataFrame."

In [147…

```
import pandas as pd
```

Vishal Acharya

```python
df = pd.DataFrame({
    "Student":["A","B","C","D"],
    "Math":[70,45,90,60],
    "Science":[75,40,85,55]
})

# Create Average column using apply row-wise
df["Average"] = df.apply(lambda x: (x["Math"] + x["Science"]) / 2, axis=1)

# Function to assign Grade
def grade(avg):
    if avg >= 80:
        return "Distinction"
    elif avg >= 60:
        return "First"
    else:
        return "Second"

# Create Grade column
df["Grade"] = df["Average"].apply(grade)

# Display final DataFrame
print(df)
```

```
  Student  Math  Science  Average        Grade
0       A    70       75     72.5        First
1       B    45       40     42.5       Second
2       C    90       85     87.5  Distinction
3       D    60       55     57.5       Second
```

70 "import pandas as pd

df = pd.DataFrame({ ""Marks"":[""50"",""60"",""Absent"",""70"",""Absent""] }) Replace ""Absent"" with 0. Convert the column to integer type. Print mean marks. Print number of students scoring above 50."

In [148…

```python
import pandas as pd

df = pd.DataFrame({
    "Marks":["50","60","Absent","70","Absent"]
```

Vishal Acharya

```
})

# Replace "Absent" with 0
df["Marks"] = df["Marks"].replace("Absent", 0)

# Convert column to integer
df["Marks"] = df["Marks"].astype(int)

# Print mean marks
print("Mean Marks:", df["Marks"].mean())

# Number of students scoring above 50
print("Students scoring above 50:", (df["Marks"] > 50).sum())
```

```
Mean Marks: 36.0
Students scoring above 50: 2
```

71 "import pandas as pd

df1 = pd.DataFrame({ ""ID"":[1,2,3], ""Name"":[""A"",""B"",""C""] })

df2 = pd.DataFrame({ ""ID"":[2,3,4], ""Marks"":[80,90,70] }) Concatenate row-wise using outer join. Count total missing values. Sort the result by ID."

In [149…

```
import pandas as pd

df1 = pd.DataFrame({
    "ID":[1,2,3],
    "Name":["A","B","C"]
})

df2 = pd.DataFrame({
    "ID":[2,3,4],
    "Marks":[80,90,70]
})

# Concatenate row-wise using outer join
df = pd.concat([df1, df2], axis=0, join="outer")
```

Vishal Acharya

```
# Count total missing values
missing_values = df.isnull().sum().sum()
print("Total Missing Values:", missing_values)

# Sort result by ID
df_sorted = df.sort_values(by="ID")

# Display final DataFrame
print(df_sorted)
```

```
Total Missing Values: 6
   ID Name  Marks
0   1    A    NaN
1   2    B    NaN
0   2  NaN   80.0
2   3    C    NaN
1   3  NaN   90.0
2   4  NaN   70.0
```

72 "import pandas as pd

students = pd.DataFrame({ ""ID"":[1,2,3,4], ""Name"":[""A"",""B"",""C"",""D""] }) marks = pd.DataFrame({ ""ID"":[2,3,5], ""Score"":[85,90,75] }) Perform outer merge using indicator=True. Count: Students present in both tables Only in students Only in marks"

In [150…

```
import pandas as pd

students = pd.DataFrame({
    "ID":[1,2,3,4],
    "Name":["A","B","C","D"]
})

marks = pd.DataFrame({
    "ID":[2,3,5],
    "Score":[85,90,75]
})

# Outer merge with indicator
df = pd.merge(students, marks, on="ID", how="outer", indicator=True)
```

Vishal Acharya

```
print(df)

# Count categories
both = (df["_merge"] == "both").sum()
only_students = (df["_merge"] == "left_only").sum()
only_marks = (df["_merge"] == "right_only").sum()

print("Students present in both tables:", both)
print("Only in students table:", only_students)
print("Only in marks table:", only_marks)
```

```
   ID Name  Score      _merge
0   1    A    NaN   left_only
1   2    B   85.0        both
2   3    C   90.0        both
3   4    D    NaN   left_only
4   5  NaN   75.0  right_only
Students present in both tables: 2
Only in students table: 2
Only in marks table: 1
```

73 "import pandas as pd

df = pd.DataFrame({ ""Dept"":[""IT"",""IT"",""HR"",""HR"",""Finance"",""Finance""], ""Salary"":[50000,70000,40000,60000,80000,75000] }) Group by Dept. Calculate mean, median, and max salary using agg()."

In [151…

```
import pandas as pd

df = pd.DataFrame({
    "Dept":["IT","IT","HR","HR","Finance","Finance"],
    "Salary":[50000,70000,40000,60000,80000,75000]
})

# Group by Dept and calculate mean, median, and max salary
result = df.groupby("Dept")["Salary"].agg(["mean","median","max"])

print(result)
```

```
            mean   median    max
Dept
Finance  77500.0  77500.0  80000
HR       50000.0  50000.0  60000
IT       60000.0  60000.0  70000
```

74 "import pandas as pd

df = pd.DataFrame({ ""Team"":[""A"",""A"",""A"",""B"",""B""], ""Runs"":[50,70,80,40,60] }) Calculate range (max - min) per team using groupby + apply."

In [152…

```python
import pandas as pd

df = pd.DataFrame({
    "Team":["A","A","A","B","B"],
    "Runs":[50,70,80,40,60]
})

# Calculate range (max - min) per team using groupby + apply
result = df.groupby("Team")["Runs"].apply(lambda x: x.max() - x.min())

print(result)
```

```
Team
A    30
B    20
Name: Runs, dtype: int64
```

75 "import pandas as pd

df = pd.DataFrame({ ""Salary"":[20000,22000,21000,25000,24000,300000] }) Detect outliers using IQR method. Remove outliers. Print cleaned dataset"

In [153…

```python
import pandas as pd

df = pd.DataFrame({
    "Salary":[20000,22000,21000,25000,24000,300000]
})
```

Vishal Acharya

Vishal Acharya

```python
# Calculate Q1 and Q3
Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)

# Calculate IQR
IQR = Q3 - Q1

# Define bounds
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Detect outliers
outliers = df[(df["Salary"] < lower) | (df["Salary"] > upper)]
print("Outliers:")
print(outliers)

# Remove outliers
clean_df = df[(df["Salary"] >= lower) & (df["Salary"] <= upper)]

# Print cleaned dataset
print("\nCleaned Dataset:")
print(clean_df)
```

```
Outliers:
   Salary
5  300000

Cleaned Dataset:
   Salary
0   20000
1   22000
2   21000
3   25000
4   24000
```

76 "import pandas as pd

df = pd.DataFrame({ ""Dept"":[""IT"",""IT"",""HR"",""HR"",""Finance"",""Finance""], ""Employee"":[""A"",""B"",""C"",""D"",""E"",""F""], ""City"": [""X"",""X"",""Y"",""Z"",""X"",""Y""] }) 3Find number of unique cities per department. Take one random employee per department. Display

results."

In [156…

```
import pandas as pd

df = pd.DataFrame({
    "Dept":["IT","IT","HR","HR","Finance","Finance"],
    "Employee":["A","B","C","D","E","F"],
    "City":["X","X","Y","Z","X","Y"]
})

# 1️⃣ Number of unique cities per department
unique_cities = df.groupby("Dept")["City"].nunique()

print("Unique Cities per Department:")
print(unique_cities)

# 2️⃣ Take one random employee per department
random_employee = df.groupby("Dept").sample(1)

print("\nOne Random Employee per Department:")
print(random_employee)
```

```
Unique Cities per Department:
Dept
Finance    2
HR         2
IT         1
Name: City, dtype: int64

One Random Employee per Department:
      Dept Employee City
4  Finance        E    X
2       HR        C    Y
0       IT        A    X
```

77 "import pandas as pd

import pandas as pd

Vishal Acharya

df = pd.DataFrame({ ""Product"":[""P1"",""P2"",""P3"",""P4"",""P5""], ""Price"":[100,200,150,300,250], ""Quantity"":[5,2,4,1,3] }) Create a new column TotalAmount = Price × Quantity using apply() row-wise. Sort the DataFrame by TotalAmount in descending order. Display the final DataFrame"

In [157…

```
import pandas as pd

df = pd.DataFrame({
    "Product":["P1","P2","P3","P4","P5"],
    "Price":[100,200,150,300,250],
    "Quantity":[5,2,4,1,3]
})

# Create TotalAmount column using apply() row-wise
df["TotalAmount"] = df.apply(lambda x: x["Price"] * x["Quantity"], axis=1)

# Sort DataFrame by TotalAmount in descending order
df_sorted = df.sort_values(by="TotalAmount", ascending=False)

# Display final DataFrame
print(df_sorted)
```

```
  Product  Price  Quantity  TotalAmount
4      P5    250         3          750
2      P3    150         4          600
0      P1    100         5          500
1      P2    200         2          400
3      P4    300         1          300
```

78 "import pandas as pd

df = pd.DataFrame({ ""Employee"":[""A"",""B"",""C"",""D"",""E""], ""Dept"":[""IT"",""HR"",""IT"",""Finance"",""HR""], ""Salary"": [50000,40000,70000,80000,45000] }) Group employees by Department. Calculate total salary and average salary per department. Display departments where average salary is greater than 50,000."

In [158…

```
import pandas as pd

df = pd.DataFrame({
    "Employee":["A","B","C","D","E"],
```

Vishal Acharya

Vishal Acharya

```
    "Dept":["IT","HR","IT","Finance","HR"],
    "Salary":[50000,40000,70000,80000,45000]
})

# Group by Department and calculate total and average salary
result = df.groupby("Dept")["Salary"].agg(["sum","mean"])

# Filter departments where average salary > 50000
filtered = result[result["mean"] > 50000]

print(filtered)
```

```
            sum     mean
Dept
Finance   80000  80000.0
IT       120000  60000.0
```

79 "import pandas as pd

df = pd.DataFrame({ ""Dept"":[""IT"",""IT"",""HR"",""HR"",""Finance"",""Finance""], ""Employee"":[""A"",""B"",""C"",""D"",""E"",""F""], ""Experience"":[2,5,3,7,4,6] }) Find the number of employees per department using size(). Find the employee with highest experience in each department using nth() or sorting. Display results."

In [159…

```
import pandas as pd

df = pd.DataFrame({
    "Dept":["IT","IT","HR","HR","Finance","Finance"],
    "Employee":["A","B","C","D","E","F"],
    "Experience":[2,5,3,7,4,6]
})

# 1️⃣ Number of employees per department
emp_count = df.groupby("Dept").size()
print("Employees per Department:")
print(emp_count)

# 2️⃣ Employee with highest experience in each department
top_emp = df.sort_values("Experience", ascending=False).groupby("Dept").nth(0)
```

```
print("\nEmployee with Highest Experience per Department:")
print(top_emp)
```

```
Employees per Department:
Dept
Finance    2
HR         2
IT         2
dtype: int64

Employee with Highest Experience per Department:
      Dept Employee  Experience
3       HR        D           7
5  Finance        F           6
1       IT        B           5
```

1. Load the dataset into a pandas DataFrame (data_result.csv) and answer the following questions.

In [18]:
```
df=pd.read_csv("data_result.csv")
```

2. View the first few rows of the dataset

In [19]:
```
df.head()
```

Out[19]:

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
0	8.11	7.68	7.11	7.43	8.18	115	Female	17020	16
1	6.48	5.90	4.15	4.29	4.96	115	Male	17021	16
2	8.41	8.24	7.52	8.25	7.75	115	Female	17022	16
3	7.33	6.83	6.33	6.79	6.89	115	Male	17023	16
4	7.89	7.34	7.22	7.32	7.46	115	Male	17024	16

3. View the last few rows of the dataset

In [20]:
```
df.tail()
```

Vishal Acharya

Out[20]:

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
41	6.30	6.24	5.85	6.36	7.00	115	Male	17061	16
42	7.78	6.93	7.44	7.86	8.21	115	Male	17062	16
43	8.22	6.66	7.07	7.29	7.61	115	Male	17063	16
44	7.67	7.07	7.04	7.07	6.93	115	Male	17064	16
45	8.41	7.59	7.41	7.89	8.11	115	Male	17065	16

4. Check the shape of the dataset

In [21]:
```
df.shape
```

Out[21]: (46, 9)

5. View the first last rows of the dataset

In [22]:
```
df.tail()
```

Out[22]:

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
41	6.30	6.24	5.85	6.36	7.00	115	Male	17061	16
42	7.78	6.93	7.44	7.86	8.21	115	Male	17062	16
43	8.22	6.66	7.07	7.29	7.61	115	Male	17063	16
44	7.67	7.07	7.04	7.07	6.93	115	Male	17064	16
45	8.41	7.59	7.41	7.89	8.11	115	Male	17065	16

6. Get summary statistics of numerical columns

In [23]:
```
df.describe()
```

Vishal Acharya

Out[23]:

	1st	2nd	3rd	4th	5th	College Code	Roll no.	Subject Code
count	46.000000	46.000000	46.000000	46.000000	46.000000	46.0	46.000000	46.0
mean	7.397609	6.930217	6.703043	7.237826	7.527609	115.0	17042.500000	16.0
std	0.798391	0.910425	0.917324	1.057981	0.967963	0.0	13.422618	0.0
min	5.670000	4.280000	4.150000	4.290000	4.860000	115.0	17020.000000	16.0
25%	6.787500	6.350000	6.217500	6.650000	6.890000	115.0	17031.250000	16.0
50%	7.440000	6.810000	7.000000	7.290000	7.625000	115.0	17042.500000	16.0
75%	8.040000	7.590000	7.322500	7.890000	8.210000	115.0	17053.750000	16.0
max	8.890000	8.720000	8.370000	9.250000	9.000000	115.0	17065.000000	16.0

7. Get summary statistics of numerical columns with 0.58 and 0.87 percentiles

In [24]:
```
df.describe(percentiles=[0.58,0.87])
```

Out[24]:

	1st	2nd	3rd	4th	5th	College Code	Roll no.	Subject Code
count	46.000000	46.000000	46.000000	46.000000	46.000000	46.0	46.000000	46.0
mean	7.397609	6.930217	6.703043	7.237826	7.527609	115.0	17042.500000	16.0
std	0.798391	0.910425	0.917324	1.057981	0.967963	0.0	13.422618	0.0
min	5.670000	4.280000	4.150000	4.290000	4.860000	115.0	17020.000000	16.0
50%	7.440000	6.810000	7.000000	7.290000	7.625000	115.0	17042.500000	16.0
58%	7.787000	6.944000	7.114000	7.647000	7.860000	115.0	17046.100000	16.0
87%	8.220000	8.000000	7.530500	8.336500	8.531500	115.0	17059.150000	16.0
max	8.890000	8.720000	8.370000	9.250000	9.000000	115.0	17065.000000	16.0

8. Get summary statistics of all types of columns

In [25]:
```
df.describe(include="all")
```

Vishal Acharya

Out[25]:

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
count	46.000000	46.000000	46.000000	46.000000	46.000000	46.0	46	46.000000	46.0
unique	NaN	NaN	NaN	NaN	NaN	NaN	2	NaN	NaN
top	NaN	NaN	NaN	NaN	NaN	NaN	Male	NaN	NaN
freq	NaN	NaN	NaN	NaN	NaN	NaN	38	NaN	NaN
mean	7.397609	6.930217	6.703043	7.237826	7.527609	115.0	NaN	17042.500000	16.0
std	0.798391	0.910425	0.917324	1.057981	0.967963	0.0	NaN	13.422618	0.0
min	5.670000	4.280000	4.150000	4.290000	4.860000	115.0	NaN	17020.000000	16.0
25%	6.787500	6.350000	6.217500	6.650000	6.890000	115.0	NaN	17031.250000	16.0
50%	7.440000	6.810000	7.000000	7.290000	7.625000	115.0	NaN	17042.500000	16.0
75%	8.040000	7.590000	7.322500	7.890000	8.210000	115.0	NaN	17053.750000	16.0
max	8.890000	8.720000	8.370000	9.250000	9.000000	115.0	NaN	17065.000000	16.0

9. Information of all columns

In [26]:
```
df.info()
```

Vishal Acharya

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 46 entries, 0 to 45
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   1st           46 non-null     float64
 1   2nd           46 non-null     float64
 2   3rd           46 non-null     float64
 3   4th           46 non-null     float64
 4   5th           46 non-null     float64
 5   College Code  46 non-null     int64
 6   Gender        46 non-null     object
 7   Roll no.      46 non-null     int64
 8   Subject Code  46 non-null     int64
dtypes: float64(5), int64(3), object(1)
memory usage: 3.4+ KB
```

10. Check for missing values

In [27]:
```
df.isnull().sum()
```

Out[27]:
```
1st             0
2nd             0
3rd             0
4th             0
5th             0
College Code    0
Gender          0
Roll no.        0
Subject Code    0
dtype: int64
```

11. Removing duplicates if duplicates

In [28]:
```
df.drop_duplicates(inplace=True)
```

12. List out female students who have greater than 7 spi in all semesters.

In [31]:
```
df[(df["Gender"]=="Female")&(df["1st"]>7)&(df["2nd"]>7)&(df["3rd"]>7)&(df["4th"]>7)&(df["5th"]>7)]
```

Vishal Acharya

Out[31]:

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
0	8.11	7.68	7.11	7.43	8.18	115	Female	17020	16
2	8.41	8.24	7.52	8.25	7.75	115	Female	17022	16
21	8.33	8.72	7.81	8.04	8.93	115	Female	17041	16
31	8.89	8.31	7.30	9.25	8.50	115	Female	17051	16

13. Find number of students those who have greater than 8 spi in all 5 semesters.

In [32]:
```
df[(df["1st"]>8)&(df["2nd"]>8)&(df["3rd"]>8)&(df["4th"]>8)&(df["5th"]>8)].shape[0]
```

Out[32]: 1

14. Find outliers of sem 4 result. Also represent statistical analysis with visualization.(boxplot)

In [37]:
```
target_column = df.columns[3]  # Access the 4th column

# Calculate quartiles for IQR outlier detection
Q1 = df[target_column].quantile(0.25)
Q3 = df[target_column].quantile(0.75)
IQR = Q3 - Q1

# Define outlier boundaries based on IQR and threshold (1.5)
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter DataFrame using boolean indexing and query
df_filtered = df[(df[target_column] < lower_bound) | (df[target_column] > upper_bound)]

print(df.shape)
print(df_filtered.shape)
```

```
(46, 9)
(1, 9)
```

In [36]:
```
df.describe()
```

Vishal Acharya

Out[36]:

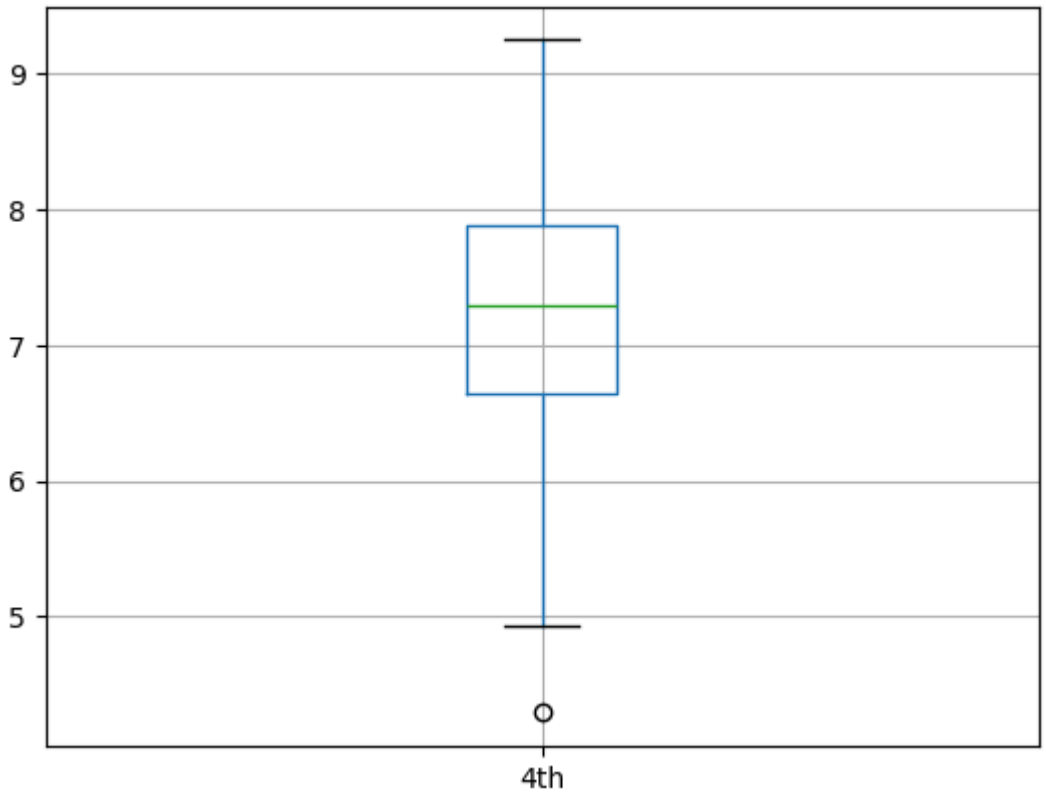
	1st	2nd	3rd	4th	5th	College Code	Roll no.	Subject Code
count	46.000000	46.000000	46.000000	46.000000	46.000000	46.0	46.000000	46.0
mean	7.397609	6.930217	6.703043	7.237826	7.527609	115.0	17042.500000	16.0
std	0.798391	0.910425	0.917324	1.057981	0.967963	0.0	13.422618	0.0
min	5.670000	4.280000	4.150000	4.290000	4.860000	115.0	17020.000000	16.0
25%	6.787500	6.350000	6.217500	6.650000	6.890000	115.0	17031.250000	16.0
50%	7.440000	6.810000	7.000000	7.290000	7.625000	115.0	17042.500000	16.0
75%	8.040000	7.590000	7.322500	7.890000	8.210000	115.0	17053.750000	16.0
max	8.890000	8.720000	8.370000	9.250000	9.000000	115.0	17065.000000	16.0

In [38]:
```
df.boxplot("4th")
```

Out[38]:
```
<Axes: >
```

Vishal Acharya

Vishal Acharya



"Use the file movies.csv which contains 1629 rows and 18 columns. Read this csv file and display the basic information like memory and data types for this data frame.

Write python code for the following cases:

1. List out Movies Released in Year 2019.
2. How Many Movies are having IMDB Rating > 7 (Display Number of Movies).
3. List out the Movies with ‘title’ and ‘story’ whose IMDB Votes > 20000.
4. List out Movies Released in Year 2018, Display only Movie Title with Release Date of Year 2018 Movies.

5. Display only Movie Title with its Wikipedia Link."

1. List out Movies Released in Year 2019.

In [139…

```
df.columns
```

Out[139…

```
Index(['title_x', 'imdb_id', 'poster_path', 'wiki_link', 'title_y',
       'original_title', 'is_adult', 'year_of_release', 'runtime', 'genres',
       'imdb_rating', 'imdb_votes', 'story', 'summary', 'tagline', 'actors',
       'wins_nominations', 'release_date'],
      dtype='object')
```

In [141…

```
import pandas as pd

# Load the dataset into a DataFrame
df = pd.read_csv("movies.csv")
# 1. List out Movies Released in Year 2019
# 1. List out Movies Released in Year 2019
movies_2019 = df[df['year_of_release'] == 2019]
print("\nMovies Released in Year 2019:")
print(movies_2019.shape[0])
print(movies_2019)
```

Vishal Acharya

Vishal Acharya

```
Movies Released in Year 2019:
75
                                  title_x     imdb_id  \
0                 Uri: The Surgical Strike   tt8291224
1                            Battalion 609   tt9472208
2     The Accidental Prime Minister (film)   tt6986710
3                         Why Cheat India   tt8108208
6                           Fraud Saiyaan   tt5013008
...                                   ...         ...
76                       Commando 3 (film)   tt8983168
77                              Mardaani 2   tt5668770
78                               Dabangg 3   tt7059844
79                              Good Newwz   tt8504014
1627                                 Daaka  tt10833860

                                            poster_path  \
0     https://upload.wikimedia.org/wikipedia/en/thum...
1                                                   NaN
2     https://upload.wikimedia.org/wikipedia/en/thum...
3     https://upload.wikimedia.org/wikipedia/en/thum...
6     https://upload.wikimedia.org/wikipedia/en/thum...
...                                                 ...
76    https://upload.wikimedia.org/wikipedia/en/thum...
77    https://upload.wikimedia.org/wikipedia/en/thum...
78    https://upload.wikimedia.org/wikipedia/en/thum...
79                                                  NaN
1627  https://upload.wikimedia.org/wikipedia/en/thum...

                                              wiki_link  \
0     https://en.wikipedia.org/wiki/Uri:_The_Surgica...
1           https://en.wikipedia.org/wiki/Battalion_609
2     https://en.wikipedia.org/wiki/The_Accidental_P...
3         https://en.wikipedia.org/wiki/Why_Cheat_India
6           https://en.wikipedia.org/wiki/Fraud_Saiyaan
...                                                 ...
76      https://en.wikipedia.org/wiki/Commando_3_(film)
77             https://en.wikipedia.org/wiki/Mardaani_2
78              https://en.wikipedia.org/wiki/Dabangg_3
79             https://en.wikipedia.org/wiki/Good_Newwz
1627                https://en.wikipedia.org/wiki/Daaka
```

Vishal Acharya

```
                           title_y                original_title  is_adult  \
0          Uri: The Surgical Strike      Uri: The Surgical Strike         0
1                      Battalion 609                 Battalion 609         0
2     The Accidental Prime Minister  The Accidental Prime Minister         0
3                   Why Cheat India                Why Cheat India         0
6                     Fraud Saiyaan                  Fraud Saiyyan         0
...                             ...                            ...       ...
76                       Commando 3                     Commando 3         0
77                       Mardaani 2                     Mardaani 2         0
78                        Dabangg 3                      Dabangg 3         0
79                       Good Newwz                     Good Newwz         0
1627                          Daaka                          Daaka         0

      year_of_release runtime                genres  imdb_rating  imdb_votes  \
0                2019     138       Action|Drama|War          8.4       35112
1                2019     131                   War          4.1          73
2                2019     112        Biography|Drama          6.1        5549
3                2019     121            Crime|Drama          6.0        1891
6                2019     109           Comedy|Drama          4.2         504
...               ...     ...                    ...          ...         ...
76               2019     133        Action|Thriller          5.7        2400
77               2019     103  Action|Drama|Thriller          7.4        8100
78               2019     160                 Action          3.0       15000
79               2019     134   Comedy|Drama|Romance          6.9       16000
1627             2019     136                 Action          7.4          38

                                                story  \
0     Divided over five chapters  the film chronicle...
1     The story revolves around a cricket match betw...
2     Based on the memoir by Indian policy analyst S...
3     The movie focuses on existing malpractices in ...
6     Fraud Saiyyan is the story of a con artist in ...
...                                                ...
76    A mysterious man is on an impending mission to...
77    Shivani Shivaji Roy locks horns with the devil...
78        Third installment of the Dabanng film series.
79    It is an upcoming Indian romantic comedy film ...
1627  Shinda tries robbing a bank so he can be wealt...

                                              summary tagline  \
0     Indian army special forces execute a covert op...     NaN
```

Vishal Acharya

```
1     The story of Battalion 609 revolves around a c...     NaN
2     Explores Manmohan Singh's tenure as the Prime ...     NaN
3     The movie focuses on existing malpractices in ...     NaN
6     Fraud Saiyyan is the story of a con artist in ...     NaN
...                                                  ...     ...
76    Commando 3 is a Hindi movie starring Vidyut Ja...     NaN
77    Shivani Shivaji Roy locks horns with the devil...     NaN
78        Third installment of the Dabanng film series.     NaN
79    It is an upcoming Indian romantic comedy film ...     NaN
1627  Shinda tries robbing a bank so he can be wealt...     NaN

                                                actors wins_nominations  \
0     Vicky Kaushal|Paresh Rawal|Mohit Raina|Yami Ga...           4 wins
1     Vicky Ahuja|Shoaib Ibrahim|Shrikant Kamat|Elen...              NaN
2     Anupam Kher|Akshaye Khanna|Aahana Kumra|Atul S...              NaN
3     Emraan Hashmi|Shreya Dhanwanthary|Snighdadeep ...              NaN
6     Arshad Warsi|Saurabh Shukla|Flora Saini|Sara L...              NaN
...                                                  ...              ...
76    Vidyut Jammwal|Adah Sharma|Angira Dhar|Sumeet ...              NaN
77             Rani Mukerji|Rajesh Sharma|Shruti Bapna|              NaN
78    Salman Khan|Sonakshi Sinha|Sudeep|Mahesh Manjr...              NaN
79    Akshay Kumar|Kareena Kapoor|Kiara Advani|Tisca...              NaN
1627                         Gippy Grewal|Zareen Khan|              NaN

                 release_date
0       11 January 2019 (USA)
1     11 January 2019 (India)
2       11 January 2019 (USA)
3       18 January 2019 (USA)
6     18 January 2019 (India)
...                       ...
76                        NaN
77                        NaN
78                        NaN
79                        NaN
1627    1 November 2019 (USA)

[75 rows x 18 columns]
```

2. How Many Movies are having IMDB Rating > 7 (Display Number of Movies).

In [142…

```
# 2. How Many Movies are having IMDB Rating > 7 (Display Number of Movies)
high_rating_movies = df[df['imdb_rating'] > 7]
high_rating_movies_count = high_rating_movies.shape[0]
print(f"\nNumber of Movies with IMDB Rating > 7: {high_rating_movies_count}")
```

```
Number of Movies with IMDB Rating > 7: 331
```

3. List out the Movies with ‘title’ and ‘story’ whose IMDB Votes > 20000

In [143…

```
# 3. List out the Movies with ‘title_x’ and ‘story’ whose IMDB Votes > 20000
popular_movies = df[df['imdb_votes'] > 20000][['title_x', 'story']]
print("\nMovies with IMDB Votes > 20000 (Title and Story):")
print(popular_movies)
```

Vishal Acharya

Vishal Acharya

```
Movies with IMDB Votes > 20000 (Title and Story):
                          title_x  \
0        Uri: The Surgical Strike
11                      Gully Boy
36                    Kabir Singh
74                    Dil Bechara
88                      Padmaavat
...                           ...
1490     Devdas (2002 Hindi film)
1565  Kabhi Khushi Kabhie Gham...
1567                       Lagaan
1568                       Lagaan
1571               Dil Chahta Hai

                                                story
0     Divided over five chapters  the film chronicle...
11    Gully Boy is a film about a 22-year-old boy "M...
36    This Sandeep Vanga directorial is a remake of ...
74              A love story about two cancer patients.
88    This fictional story is set in 13th century me...
...                                                 ...
1490  Devdas Mukherji is black-listed by his multi-m...
1565  Yashvardhan Raichand lives a very wealthy life...
1567  This is the story about the resilience shown b...
1568  This is the story about the resilience shown b...
1571  Three young men Akash  Sameer  and Siddharth a...

[105 rows x 2 columns]
```

In [ ]:

```
4. List out Movies Released in Year 2018, Display only Movie Title with Release Date of Year 2018 Movies.
```

In [144…

```
#  List out Movies Released in Year 2018, Display only Movie Title with Release Date of Year 2018 Movies
movies_2018 = df[df['year_of_release'] == 2018][['title_x', 'release_date']]
print("\nMovies Released in Year 2018 (Title and Release Date):")
print(movies_2018)
```

```
Movies Released in Year 2018 (Title and Release Date):
                        title_x              release_date
4               Evening Shadows   11 January 2019 (India)
5                   Soni (film)     18 January 2019 (USA)
16       Mard Ko Dard Nahi Hota       22 March 2019 (USA)
17                 Hamid (film)     15 March 2019 (India)
20    Mere Pyare Prime Minister     15 March 2019 (India)
...                         ...                       ...
156                Rajma Chawal  30 November 2018 (India)
157            Zero (2018 film)    21 December 2018 (USA)
158                      Simmba    28 December 2018 (USA)
166          Thugs of Hindostan     8 November 2018 (USA)
1626            Sabse Bada Sukh                       NaN

[79 rows x 2 columns]
```

In [ ]:

```
5. Display only Movie Title with its Wikipedia Link."
```

In [145…

```
# 5. Display only Movie Title with its Wikipedia Link
movie_wikipedia_links = df[['title_x', 'wiki_link']]
print("\nMovie Titles with Wikipedia Links:")
print(movie_wikipedia_links)
```

Vishal Acharya

```
Movie Titles with Wikipedia Links:
                                  title_x  \
0                Uri: The Surgical Strike
1                            Battalion 609
2     The Accidental Prime Minister (film)
3                        Why Cheat India
4                        Evening Shadows
...                                  ...
1624                Tera Mera Saath Rahen
1625                 Yeh Zindagi Ka Safar
1626                      Sabse Bada Sukh
1627                                Daaka
1628                             Humsafar

                                           wiki_link
0     https://en.wikipedia.org/wiki/Uri:_The_Surgica...
1           https://en.wikipedia.org/wiki/Battalion_609
2     https://en.wikipedia.org/wiki/The_Accidental_P...
3         https://en.wikipedia.org/wiki/Why_Cheat_India
4         https://en.wikipedia.org/wiki/Evening_Shadows
...                                                 ...
1624  https://en.wikipedia.org/wiki/Tera_Mera_Saath_...
1625  https://en.wikipedia.org/wiki/Yeh_Zindagi_Ka_S...
1626      https://en.wikipedia.org/wiki/Sabse_Bada_Sukh
1627                https://en.wikipedia.org/wiki/Daaka
1628             https://en.wikipedia.org/wiki/Humsafar

[1629 rows x 2 columns]
```

2. Display basic information like memory and data types for this data frame.

In [177…

```
df.info()
```

Vishal Acharya

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Height  10000 non-null  float64
 1   Weight  10000 non-null  float64
 2   Male    10000 non-null  int64
dtypes: float64(2), int64(1)
memory usage: 234.5 KB
```

3. Display basic statistics like mean, std, quartiles, etc. for this data frame.

In [178…
```
df.describe()
```

Out[178…

	Height	Weight	Male
count	10000.000000	10000.000000	10000.000000
mean	66.367560	161.440357	0.500000
std	3.847528	32.108439	0.500025
min	54.263133	64.700127	0.000000
25%	63.505620	135.818051	0.000000
50%	66.318070	161.212928	0.500000
75%	69.174262	187.169525	1.000000
max	78.998742	269.989698	1.000000

4. Create a correlation table for the data frame and comment about what kind ofcorrelation is there between Height and Weight.

In [179…
```
df.corr(numeric_only=True)
```

Vishal Acharya

Out[179…

	Height	Weight	Male
Height	1.000000	0.924756	0.691072
Weight	0.924756	1.000000	0.796723
Male	0.691072	0.796723	1.000000

In [180…

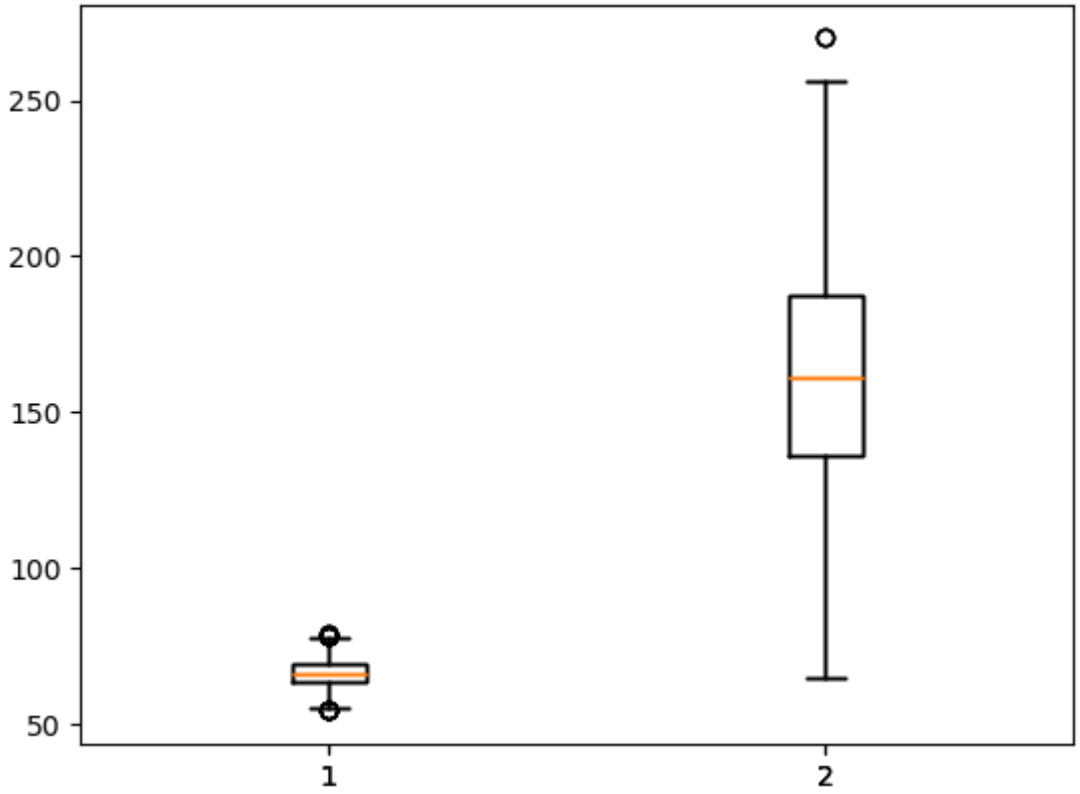
```
print("Strong relation:posative high vale")
```

```
Strong relation:posative high vale
```

5. Do Height and Weight contain any outliers? Answer by creating boxplots for both.

In [183…

```
plt.boxplot(df[["Height","Weight"]])
plt.show()
```

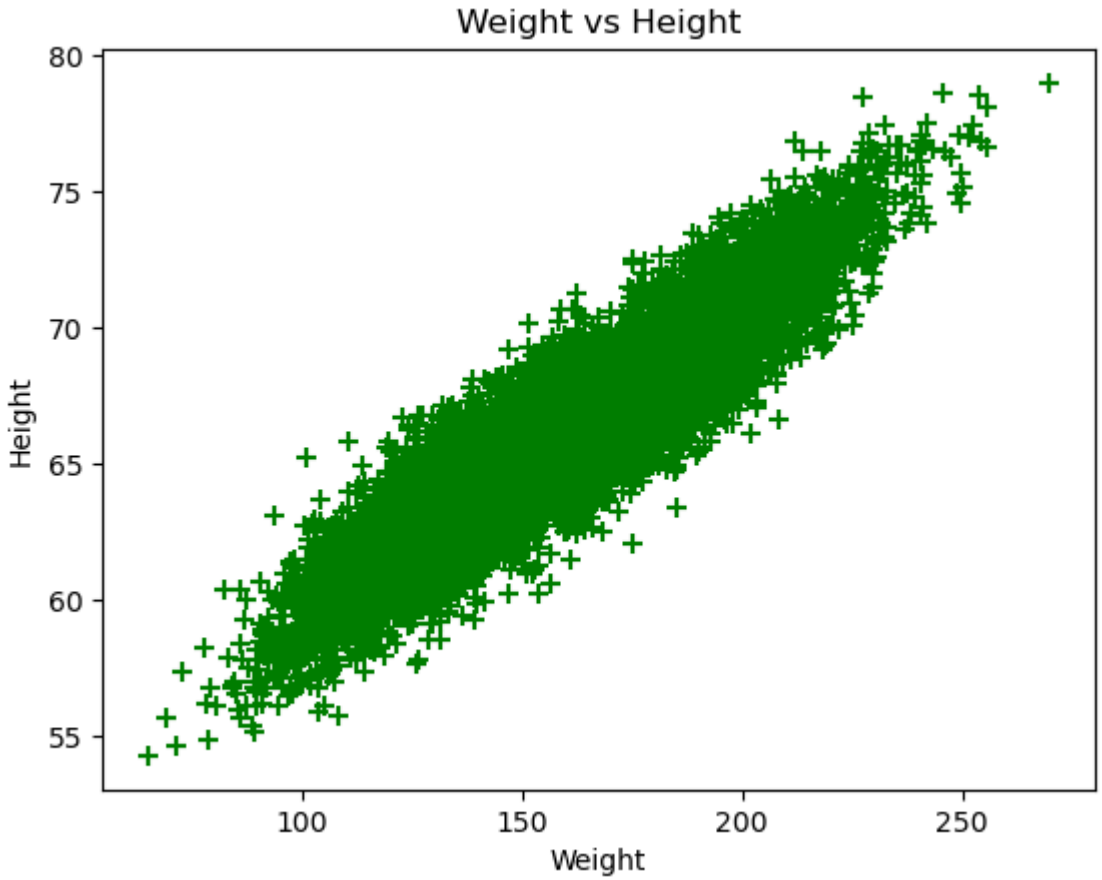


Vishal Acharya

6. Finally, create a scatter plot of Weight v/s Height with the following specifications: (i) use + sign, colour green and size 50 for markers. (ii) Label X Axis as Weight and Y Axis as Height. (iii) Display title on top as Weight vs Height

In [184…

```
plt.scatter(df['Weight'], df['Height'], marker='+', color='green', s=50)
plt.title('Weight vs Height')
plt.xlabel('Weight')
plt.ylabel('Height')
plt.show()
```



Vishal Acharya